

JOHANNES KEPLER
UNIVERSITÄT LINZ
Netzwerk für Forschung, Lehre und Praxis



TABU Search

Solving Production Planning Problems Using the Tabu Search Metaheuristic

MASTER'S THESIS

for obtaining the academic title

Master of Science

in

INTERNATIONALER UNIVERSITÄTSLEHRGANG INFORMATICS: ENGINEERING & MANAGEMENT

composed at ISI-Hagenberg

Handed in by:
Alexandra Jimborean, 0855835

Finished on: 25, June, 2009

Scientific Advisor: *Prof. Dr. Tudor Jebelean*

Technical Advisors: *Dr. Roman Stainko*, *Dr. Peter Stadelmeyer*

Hagenberg, June, 2009

ABSTRACT

Throughout the last decades, the production planning problem has generated considerable amount of research. Being an NP-hard optimization problem, the target is outlined as finding heuristic solutions, rather than optimal ones. This thesis describes the current state of the art and analyzes diverse models designed for solving the production planning problem. For the present purposes, the problem is tackled following an adapted Discrete Lot Sizing and Scheduling model, and a specifically tailored algorithm based on the Tabu Search metaheuristic is proposed. A theoretical overview of the Tabu search method, describing the key elements and the characteristic advanced strategies associated with this searching technique, is provided. A detailed description of the construction of the initial solution algorithm is offered together with the neighborhood operators involved in the search. In addition to the short-term memory based strategies, diversification and intensification mechanisms are employed to enhance the efficiency of the proposed solution method. Experimental testing is conducted to determine the most suitable values of the parameters for the Tabu Search. Finally, the paper draws a parallel by comparing the performance of the Tabu Search method and two other well-known optimization techniques.

Contents

1 Introduction **1**

1.1 Production Planning 1

1.2 Complexity - NP hard optimization problems 4

1.3 Solving Production Planning problems 6

2 Modeling Production Planning & State of the Art **7**

2.1 Lot Sizing and Scheduling 7

2.1.1 Capacity Constraints 8

2.1.2 Backlog Costs 9

2.1.3 Set-Up Costs 10

2.1.4 Multistage Problems 10

2.1.5 Multiple Products 11

2.1.6 Time Division 12

2.1.7 Multiple Machines 14

2.2 Thoughts from history 14

2.3 Tabu Search applied on Production Planning problem 17

3 Problem Description **21**

3.1 Background 21

3.2 Problem Description 22

3.2.1 Project Topic 22

3.2.2 Description 23

3.3 Mathematical Model 26
3.3.1 Goal: 26

4 TABU Search **30**
4.1 Solution Method - TABU Search 30
4.1.1 TS Basic Strategies 31
4.1.2 TS Advanced Strategies 33

5 Implementation Details **45**
5.1 Representation of the Solution and Parameters 45
5.1.1 Solution Representation: 45
5.1.2 Parameters Representation: 47
5.2 Initial Solution Construction 49
5.2.1 Construction Algorithm 49
5.2.2 Results 51
5.3 TS Short-Term Memory 52
5.3.1 Neighborhood operators 53
5.3.2 Tabu Search Algorithm 61
5.3.3 TS Advanced Strategies 63

6 Documentation of Results **67**

7 Conclusions **78**

References **81**

Chapter 1

Introduction

The economic success of modern companies is more and more decided by their ability to react quickly and effectively to changing selling conditions. They have to use their production capacities and resources in an optimal way to be a successful player on the global market. The key for this success is effective production planning. Production function encompasses the activities of procurement, allocation and utilization of resources. The main objective of production process is to produce the goods and services demanded by the customers in the most efficient and economical way. Therefore efficient management of the production function is of utmost importance in order to achieve this objective.

1.1 Production Planning

Production planning represents the beating heart of the manufacturing process. It entails the acquisition and allocation of limited resources to production activities so as to satisfy customer demand over a specified time horizon. As such, planning and control problems are inherently optimization problems, where the objective is to develop a plan that meets demand at minimum cost or that fills the demand that maximizes profit. Therefore, the main purpose is to efficiently organize the resources, minimize production costs and maximize efficiency. The underlying optimization problem will vary due to differences in the manufacturing and market context.[17, 15]

In a manufacturing environment, the production system is built up from several production cells, e.g. flow lines or work centers. The macro-structured is then subdivided in a micro-structure, where each cell can perform a series of operations. The architecture of a production system consists of several machines and input materials, components that

have to be produced and assembled with respect to a set of constraints. In order to increase performance, machines must work simultaneously and all these components are to be constructed and assembled concurrently in an optimal order, until the final product is ready for deliverance. The whole manufacturing process is triggered by customers' demands, that are to be met by the production plan. These demands may be fixed or they may vary in the time horizon considered for the planning. In many contexts, the planning takes into account also future demands, either known or forecasted.[17]

To motivate a planning activity, one has to choose the key planning decisions, which economically worth being considered. These may be related to the inventory costs, namely storing goods once they have been produced. On the other hand, production costs must be put into balance. Except the trade-off between low set-up costs, which favor large production lots, and low holding costs, one has to consider also backlog costs. Or the costs incurred for not meeting a certain demand. Another relevant cost that might be taken into consideration in some cases is the acquisition of resources. "By definition, production-planning models include decisions on production and inventory quantities. But in addition, there might be resource acquisition and allocation decision, such as adding to the work force and upgrading the training of the current work force."[17]

One problem that occurs while planning regards the use of resources and their scarce capacity. Production of one item requires a certain amount of one or more resources (e.g. machines, manpower, energy), which have a limited capacity per time unit. Additionally, producing an item may be conditioned by previous production of several other items, therefore a particular order must be imposed. Due to sharing common resources, the problem of sequencing the production lots increases in difficulty. Consider a production system that consists of a group of machines that produce different types of products. In this respect, each machine has a production capacity, which differs with respect to the type of the product. The production of a different product type on a machine is preceded by a setting, during which the machine cannot produce. Therefore, the production is delayed and a cost is associated to each setting, as part of the production costs which are aimed to be minimized. An aspect of sharing resources is that the production of such items cannot coincide, as different setup states are required.

One aspect that is worth mentioning while describing production planning refers to the time horizon considered. Also named "planning horizon", it indicates the period of time for which the planning is made. It may be given in terms of days or even months, but additional time division is performed, for instance dividing the days in shifts. As presented in the next chapter there are various modalities of dividing the time and modeling production. "The choice of planning horizon is dictated by the lead times to enact production and resource-related decisions, as well as the quality of knowledge about future

demand."[17]

Typically, production planning is done in a rolling horizon. Namely, the planning is created in advance, but only the decisions for the first periods of the the horizon are implemented. Then, the planning is revised and adapted to any changes that may occur due to the uncertainty brought by demand and production forecast. This approach offers robustness, as the planning is periodically revised and updated. For instance, a company may plan for the next half of the year, but then reconsider the planning once per month to incorporate new information on demand and production.

For highly complex problems, production planning is done at an aggregate level, for resources and for products as well. The purpose is to reduce problem complexity and for accomplishing it, different but similar products are grouped together in families. Alternatively, resources that share a set of similarities can be grouped into an aggregate resource. The production planning is done for each aggregate level, hence the problem is decomposed in less complex subproblems. However, special care is required when deciding the aggregate level, in order to ensure that the aggregate plan can be disaggregated into feasible production schedules.

Moreover in case of complex products, one must decide the level and product structure that is going to be considered for building the planning. For instance, in a number of cases, it might suffice to include only the end-items in the production planning. As a consequence, the production of subassemblies or other components will be determined by the schedule of the end items. On the other hand, disregarding the planning of the subcomponents might lead to results of lower quality in some particular contexts. In such cases, a multistage planning model must be developed, to allow the simultaneous planning of end items and their components. Nevertheless, the direct consequence will be a much larger model. [17]

Finally considering the above presented aspects, the goal would be to develop a feasible production planning. Feasibility is defined as problem dependent. But in short, the planning should "meet the demands and provide release dates and amounts for all products including components parts."[17] However, from an economical point of view, gaining feasibility is not sufficient. Hence one needs to evaluate the plans and this can be achieved by means of an objective function. It is aimed to measure the costs, for example holding costs, production costs, backlog costs and to minimize them. Another variant would be to maximize production. Either case, the final goal is to find a feasible production plan with an optimum objective function value.

To conclude, even small instances of production planning problems might be very complex

and difficult to solve to optimality. As the problem size increases, or features like dynamic demands or dependencies between products are considered, even building a feasible plan will turn into an NP hard optimization problem, as presented in the following section.

1.2 Complexity - NP hard optimization problems

When dealing with complexity, two classes are intensively discussed and their equivalence relation remained an unsolved question in the theory of computer science: P and NP classes.

Problems belonging to class P can be solved in a polynomial time, with respect to the size of the input. NP problems are those considered more difficult to be solved, but a solution to such a problem can be verified in polynomial time. Many attempts for solving NP problems in polynomial time have been made, but without any success. This led the scientist to believe that for solving such problems superpolynomial time is required. However, one of the most challenging open questions in computer science theory is whether P = NP ? The abbreviation NP stands for "Nondeterministic Polynomial time". It represents the class of problems for which a solution can be verified in polynomial time on a deterministic machine, or, equivalently, it can be solved in polynomial time on a non-deterministic machine. A deterministic machine has the property that considering the same state and the same input data, the generated steps will always be the same. Therefore, solving an NP problem follows two steps:

1. Guess solution in the first phase
2. In the second phase, the solution is verified in polynomial time

In this context, one of the important NP subclasses is represented by the NP-complete problems. Besides the properties inherited from the NP class, NP-complete problems have the additional feature that any problem in the NP complete class can be reduced to another NP complete problem in polynomial time. One says that a problem Π is polynomially reducible to a problem Π', if a polynomial algorithm exists, which transforms each instance of Π in an instance of Π' , such that each instance of Π returns "yes" if and only if "yes" is also returned for the corresponding instance of Π'.

In relation to NP and NP-complete problems, one defines the class of NP-hard problems, informally known as "at least as hard as any NP-problem, although it might, in fact, be harder." Hence, "a problem is NP-hard if an algorithm for solving it can be translated

into one for solving any NP-problem (nondeterministic polynomial time) problem."[18] A more illustrative view is offered by the following Figure 1.1:

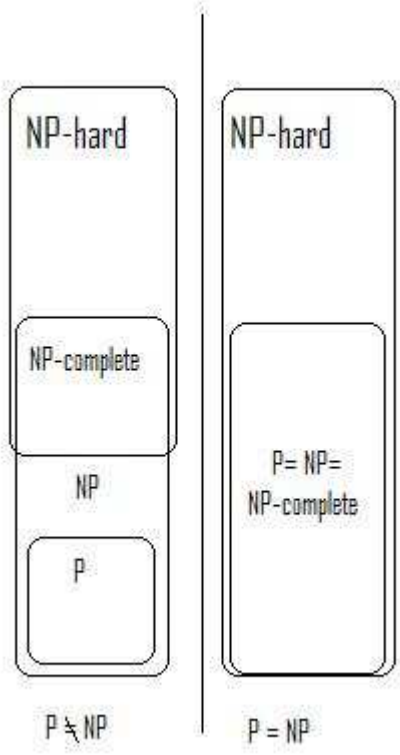


Figure 1.1: P & NP

The NP-family naming system is confusing: NP-hard problems are not all NP, despite having 'NP' as the prefix of their class name. However, the names are now entrenched and unlikely to change. On the other hand, the NP-naming system has some deeper sense, because the NP family is defined in relation to the class NP: NP-complete means problems that are 'complete' in NP, i.e. the most difficult to solve in NP; NP-hard stands for 'at least' as hard as NP (but not necessarily in NP); NP-easy stands for 'at most' as hard as NP (but not necessarily in NP); NP-equivalent means equally difficult as NP, (but not necessarily in NP). [13]

Due to the increasing complexity of the production planning problem with respect to its size, the problem enters the category of NP-hard problems and an advanced algorithm for finding a nearly optimal solution is required.

1.3 Solving Production Planning problems

In the production planning environment, considering only one machine, a fixed demand and a simplified set of constraints, finding the optimal planning already represents an NP-hard problem. Although the planning might be made by hand by a human expert, it would require a considerable amount of time, experience and intuition. Additionally, there will be no indication regarding the quality of the plan.

But for real-life problems, the number of products is increasing, while the demands for each type are decreasing, therefore creating a feasible production plan becomes a more and more difficult task for a number of machines that work in parallel. An increasing number of resources, items to be produced, demands and a full set of constrains generate problems of very high complexity, which are almost impossible to be solved by hand.

Although under some conditions, the planning can be created manually, the process of building and optimizing a production plan may take too long and it becomes economically inefficient. Many factors contribute to the modification of the input parameters of the system, resulting in the necessity of recalculating the production plan very often. Among them, one can consider that demands change over relatively short periods of time, the number of items being produced may vary from one period of time to another or resources may change, either in their number or features.

Hence, in order to construct a competitive and nearly optimal planning, a dedicated software must be developed. It is aimed to build a robust production plan and also to guarantee its quality up to a certain point. Although optimality cannot be guaranteed, the plan is continuously optimized and it is surely better than all the others considered. Moreover, all the constraints are verified automatically. And more importantly, the production planning process requires a significantly shorter amount of time compared to the manual process, which boosts the economic success of the production system.

Chapter 2

Modeling Production Planning & State of the Art

Either in the field of economics, engineering, biology or any other science, real world is flooded with hard optimization problems. Among them, the problem of scheduling and planning plays an important role, since it finds its practical applications in all domains. As a direct consequence, Production Planning optimization represents a major field of interest for researchers and diverse solution methods have been proposed. However, being an NP-hard optimization problem, it is almost impossible to prove the optimality of a found solution in reasonable time for interesting real-life problem sizes. The target of my thesis is to model a production planning problem and apply a version of the Tabu search method, customized for the specific problem. Also, the focus will be on adjusting the meta-parameters that will guide the search in order to construct nearly optimal solutions.

2.1 Lot Sizing and Scheduling

Having such a great impact, production planning problems have been formulated in many ways throughout the years. Maybe one of the most relevant model formulation is the **Lot Sizing and Scheduling** problem, which has a number of variants itself. In manufacturing environment, a lot size refers to the amount of units of a particular item that is produced on a machine after a set-up has been performed. Lot sizing and scheduling concerns the determination of the appropriate lot sizes of items that are to be produced in each period of the time horizon, such that the production costs are minimal. Since each problem is specific and unique, the model has to be flexible and adaptable to diversified complexities and to different input and output parameters. Consequently,

there are models involving different features, like single or multiple machines. "In the latter case these can be parallel machines (in a single stage), machines in sequence (i.e. multistage) or even parallel multistage machines."[5] Similarly, production of single or multiple items may be modeled. Additionally, features like set-up costs, backlogs costs or fixed or varying demands may be considered.

In its basic form, the lot sizing model can be stated as a rather simple model, formulated for only one stage with infinite capacity of given resources and a single item that has to be produced for T periods of time, in order to meet a stationary demand. Consequently, the objective function consists in minimizing the inventory holding costs from one period to the next.

Objective:

$$\min \left\{ \sum_{t \in T} h I_t \right\}. \tag{2.1}$$

Subject to the constraints:

$$I_{t-1} + q_t - I_t = d_t, \quad \forall t \in T. \tag{2.2}$$

$$0 \leq I_t, 0 \leq q_t, \quad \forall t \in T. \tag{2.3}$$

Where h represents the inventory holding costs of one product from one time period to the next and d_t is the demand for the period t. The decision variables are: q_t, which indicates the number of items that are produced in the period t, and I_t, representing the inventory at the end of each time period.

An optimal solution to this problem is easy to derive, namely, production should be equal to the demand in each time period: $q_t = d_t$, $\forall$ t $\in$ T.

Introducing additional requirements and restrictions, the model develops into a more complex one, which is suitable for the real-life problems.

2.1.1 Capacity Constraints

A first step towards a more realistic approach is to consider a finite capacity of given resources, which is translated into introducing capacity constraints, such that in each time period, production is limited by a measure C_t, the production capacity for each

period t.

$$q_t \leq C_t, \quad \forall t \in T. \tag{2.4}$$

In case the demand exceeds the capacity in a particular time period, the solution is not trivial anymore.

2.1.2 Backlog Costs

Obviously, in production planning one of the main goals is to fulfill the demands, but this cannot always be achieved due to the capacity constraints. In consequence, one has to introduce the backlogs in the model and to penalize them in the objective function. A cost will be associated to carrying a backlog of one product unit from one period to the next. Hence, the aim is now to minimize the inventory costs and the backlog costs.

The objective function changes its form from equation (1) to:

$$\min \left\{ \sum_{t \in T} h I_t^+ + g I_t^- \right\}. \tag{2.5}$$

And the constraints presented in equations (2) and (3) become

$$I_{t-1}^+ - I_{t-1}^- + q_t - I_t^+ + I_t^- = d_t, \quad \forall t \in T. \tag{2.6}$$

$$0 \leq I_t^+, 0 \leq I_t^-, 0 \leq q_t, \quad \forall t \in T. \tag{2.7}$$

including the capacity constraints (4).

In the current model, the objective function minimizes simultaneously the costs for the holding inventory and for the backlogs. The new parameter g represents the penalty cost for a backorder of one product unit, whereas I_t^+ and I_t^- are decision variables. I_t^+ and I_t^- represent the inventory and the backlog quantity at the end of period t.

Although the complexity is increased, the problem can be modeled as a linear programming problem with continuous constraints.

2.1.3 Set-Up Costs

Although various models have been developed to handle the set-ups, for the simplified version it suffices to mention that in order to produce an item in a time period a set-up must be performed first. In the formulation proposed by Wagner and Whitin (WW) [12], the capacity constraint is not considered and the problem is a single-product single-machine problem. Considering the set-up costs and the inventory costs, one has to find the optimal planning. It may be more efficient to pay some inventory costs and produce for a number of periods, than performing a set-up in every period. Accordingly, the objective function is:

$$\min \left\{ \sum_{t \in T} s y_t + h I_t \right\}. \tag{2.8}$$

Where s represents the set-up cost and y_t is a binary decision variable which takes the value 1 if a set-up has been performed in period t and 0 otherwise. Since production is possible only when the machine is correspondingly set-up, the value of y_t influences the value of q_t. Hence, if $y_t = 0$, also $q_t = 0$ and if $y_t = 1$, $q_t \geq 0$.

The new constraint is

$$q_t \leq M_t y_t, \quad \forall t \in T. \tag{2.9}$$

where M_t is un upper bound for q_t.

Due to the discrete nature of the set-up decision variables it is no longer a linear programming problem (LP), but a mixed integer linear programming problem (MILP). However, the presence of set-ups leads to many different formulations, when one or multiple set-ups are allowed per time period, if they appear in the beginning or in the end of a period or they are carried over periods or not. All in all, once the set-ups are introduced, an additional constraint has to be fulfilled. Namely, that an item can only be produced if and only if a corresponding set-up has been performed.

2.1.4 Multistage Problems

In the above presented formulations, only single-stage models have been described. More complex problems lead to multistage models. In case the machines produce in parallel, one deals with a single stage model, whereas dependencies among machines determines a multistage model formulation. This new angle decomposes the problem in different

stages or levels, having the property that each level resembles the single-stage model. Nevertheless, the production is seen as unitary, in terms that production in each level is decided considering the demand of the next level. Therefore, one has to decide the lot sizes at each level such that, combined together, they result in minimal costs. This problem is NP complete.

The terms "multistage"and "multilevel"may be considered equivalent. However, one should not confuse the notions multistage and multiple-item, also called multiple-product. "Multiple products can be multistage if the production of one item is dependent on another. Otherwise the production can be called single-stage, or single-level"[5].

2.1.5 Multiple Products

A single-product model can be easily extended to the multiple products model. All the previously described constraints and the objective function will keep their form, with the modification that the parameters and the decision variables will be now not only period but also product dependent. Hence, the objective function:

$$\min \left\{ \sum_{j \in J} \sum_{t \in T} h_{jt} I_{jt}^{+} + g_{jt} I_{jt}^{-} \right\}. \tag{2.10}$$

And the constraint:

$$I_{j(t-1)}^{+} - I_{j(t-1)}^{-} + q_{jt} - I_{jt}^{+} + I_{jt}^{-} = d_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.11}$$

In the above equations J represents the index set of items, whereas I_{jt}^{+} and I_{jt}^{-} represent the positive, respectively the negative inventory of item j at the end of period t. Similarly, the decision variable q_{jt} indicates the number of items j that are produced in period t. And d_{jt} is the demand for item j in period t. Likewise, h_{jt} and g_{jt} represent the inventory, respectively the backlog cost per unit of item j in time period t.

For a capacitated problem, one has to add the constraint:

$$\sum_{j \in J} u_j q_{jt} \leq C_t, \quad \forall t \in T. \tag{2.12}$$

Where u_j is the capacity need for one unit of product j and C_t is the total capacity of machine at period t. Combining the above ideas, one reaches another variant of the model, Capacitated Lot Sizing Problem (CLSP), described by the following equations:

Objective function:

$$\min \left\{ \sum_{j \in J} \sum_{t \in T} s_j y_{jt} + h_j I_{jt} \right\}. \tag{2.13}$$

Subject to:

$$I_{jt} = I_{j(t-1)} + q_{jt} - d_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.14}$$

$$u_j q_{jt} \leq C_t y_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.15}$$

$$\sum_{j \in J} u_j q_{jt} \leq C_t, \quad \forall t \in T. \tag{2.16}$$

$$y_{jt} \in \{0, 1\}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.17}$$

$$I_{jt}, q_{jt} \geq 0, \quad \forall j \in J, \quad \forall t \in T. \tag{2.18}$$

where s_j represents the set-up cost for item j and h_j is the holding costs of product j.

However, scheduling decisions are not integrated in this version. Therefore, the approach most often adopted is to find a solution that fulfills the constraints and solves the CLSP problem and a solution to the scheduling problem is to be found separately for each time period afterwards. Additional information regarding CLSP is provided by [5, 8]

2.1.6 Time Division

Having more than one type of products, one generates new model formulations, differentiated by the number of product types that can be produced in one time period. Hence, the well known models "Big Bucket" and "Small Bucket" have been described in literature. Following the Big Bucket formulation, the time horizon is divided in longer time periods, such that set-ups and production of several items per period are allowed. In contrast, the Small Bucket formulation concept handles production by dividing the time in such short time periods which allow the set-up and production of one or at most two product types.

The CLSP is a big bucket problem, since several items may be produced in the same period. Dividing the periods into micro-periods determines a lot sizing and scheduling model, called Discrete Lot-Sizing and Scheduling Problem (DLSP). DLSP allows production of only one item per micro period, hence it is a small bucket problem. The main difference between these models is that DLSP revolves around an *all-or-nothing* assumption, which forces the production on each machine to full capacity, while CLSP allows

machines to work under their full capacity. This difference is spotted by replacing the constraints given by the equations (15) and (16) by:

$$u_j q_{jt} = C_t x_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.19}$$

And the additional constraints:

$$\sum_{j \in J} x_{jt} \leq 1, \quad \forall t \in T. \tag{2.20}$$

$$y_{jt} \geq x_{jt} - x_{j(t-1)}. \tag{2.21}$$

where x_{jt} is a boolean variable indicating whether machine is set in time period t to produce item j ($x_{jt} = 1$) or not ($x_{jt} = 0$). The variable y_{jt} indicates whether a set-up occurs on machine for item j in period t.

The DLSP's *all-or-nothing* assumption is fairly strict, but motivated by more efficient implementations of mathematical programming approaches. However, a more realistic approach is given by Continuous Set-up Lot Sizing Problem (CSLP) which follows the DLSP model, with the difference that it relaxes constraint (19) to:

$$u_j q_{jt} \leq C_t x_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.22}$$

Therefore CSLP allows a machine to produce under its full capacity, but still only one item may be produced per period. Hence it is also a small bucket problem.

The most important drawback of CLSP is that the system does not produce in full and the remaining capacity is lost. Consequently, the direct consequence was the development of a new model, aimed to overcome this shortcoming. The Proportional Lot Sizing and Scheduling Problem (PLSP) uses the idle time of a machine to perform a set-up and start the production of a new item in the particular period. But since more than one product type can be produced in one period, the natural question regards the order in which production appears in the frame of a period. And the answer is given by the setting state of the machine. Since production is possible if it has been properly set up either at the beginning or at the end of the period. Hence, the main characteristic of PLSP is that at most two set-ups can be performed per period. A more detailed overview of this model is referenced in [8].

A variant for the Small Bucket approach is the General Lot Sizing and Scheduling Problem (GLSP), which imposes a limit for the number of micro-periods. The parameters are equivalent to the ones in DLSP, but each lot is uniquely assigned a position number that defines a sequence.

2.1.7 Multiple Machines

In order to tackle a real-life problem, in most of the situations a multiple machine model is required and in some cases a multi-stage model must be formulated. In what follows, only the single-stage multiple machine model is described, namely, machines that work in parallel. Hence for a multiple-machine model, the constraints have the form:

$$I_{jt} = I_{j(t-1)} + \sum_{m \in M} q_{mjt} - d_{jt}, \quad \forall j \in J, \quad \forall t \in T. \tag{2.23}$$

$$\sum_{j \in J} u_{mj} q_{mjt} \leq C_{mt}, \quad \forall m \in M, \quad \forall t \in T. \tag{2.24}$$

where q_{mjt} is the production of item j on machine m in period t, u_{mj} is the capacity needed for one unit of product j on machine m and C_{mt} is the capacity available on machine m in period t.

Three subproblems have been distinguished for this special case of parallel machines, namely identical parallel machines, uniform parallel machines and unrelated parallel machines. Nevertheless, models that treat the machines as a unitary system have also been developed, as indicated by [5].

All in all, the Lot Sizing and Scheduling Problem has attracted increasing research interest, resulting in good models for the production planning and scheduling problems from real life. Aiming at an optimal solution to the problem, diverse methods have been applied. Among them one can enumerate some like Mathematical Programming, Evolutionary Algorithms, Genetic Algorithms, Evolution Strategies and other Heuristics and Metaheuristics.

2.2 Thoughts from history

Planning problems are notoriously hard to solve and even on small instances many methods have encountered serious difficulties. However, although many different approaches have been developed for solving them, one of the most widely adopted and extended is the Mixed Integer Programming (MIP) method. From the class of Mathematical Programming, MIP has the underlying idea that some of the decision variables contained in the solution are constrained to take only integer values. As a consequence, the problem loses convexity becoming non-convex and extremely hard to be solved. Nevertheless, the use of integer variables greatly expands the scope of the optimization problems to be modeled and solved. As a special case, binary integer variables are used, namely integer variables

that can take values only in the set {0,1}. These kind of variables are used to model yes / no decisions. Adding more integer variables in the solution, the memory and the time necessary for solving the problem grow exponentially. This is an immediate consequence of the fact that many combinations of integer variables have to be computed and tested. In the branch-and-bound or branch-and-cut approach for mixed integer programming problems, the search space is searched using a tree structure as a guide line. For each node in this search tree a solution to a linear optimization problem has to be found. The main advantage is that MIP can find and guarantee the global optimum for a problem, or provides information about the "quality"of a found solution (given by an upper and a lower bound). This already represents an enormous achievement in the world of NP hard problems. But this comes with the price of considerably long running time, that can easily exceed days, depending on the size of the problem. And this drawback could not have been overcome by supercomputers, grid computing nor helped by sophisticated parallelization algorithms.

Since in a dynamic system, whose input parameters change their values frequently, this shortcoming would turn the method into a rather impractical one, the immediate step was to combine MIP with other methods. A hybrid solution is presented in more details in [4], where MIP and constraint programming shake hands towards more promising results. The idea is to decompose the problem in levels, as schedules for the lots and then to integrate these subschedules in order to obtain the planning. For determining the subschedules a MIP approach has been adopted, suitable for small sized problems, and the integration is based on constraint programming [4]. However, an unforseen issue is that conflicts between the subschedules appeared when integrating them, as not all the constraints could have been properly simultaneously fulfilled. Therefore, solving these conflicts turned into a bottleneck of the generation of the solution process. Other hybrid variants would use heuristics in order to find a feasible starting solution, which was then improved with MIP. However, most of these solution methods are time consuming.

Previous studies have often followed the methods of Linear Programming, Non-Linear Programming or Mixed Integer Programming, either in their pure form, or using extensions like Lagrange relaxation. But the process of generating solutions has been reported to take between 10 to 20 hours, which is unacceptable if real-life problems have to be solved in a short time. One more interesting approach was based on the famous Johnson's rule [1], which is able to produce the optimal solution in polynomial time for a two-stage scheduling problem. But no generalization can be made for a more complex problem, involving more than two machines.

Since in the area of Production Planning exact methods like Linear Programming or Mixed Integer Programming are not useful for big problem instances, a new spark had

to be brought on the production planning problems' scene. Under the name of Heuristic Optimization, a number of new methods have been applied with the aim of generating feasible, nearly optimal solutions and in a reasonable time.

One of the most common heuristic method is Genetic Algorithms (GA) [2], which have been intensively explored in the last decade. This already provides solid theoretical fundamentals and practical experience. But results offered by the literature surveyed by [11] do not consider the case of increasing customers demands. This represents a major drawback in real-world problems, which makes the approach unsuitable for production planning. Moreover, a good strategy should take into account the multitude of constraints that appear in practice and their combinations. "The combination of factors simultaneously affecting the quality of the Aggregate Production Plan is a characteristic of real-life problems and their consideration can make the difference between a purely academic treatment of the subject and a result that can be applied or transferred immediately in practice."[6] Many production planning problems involve highly complex constraints, which make them very difficult to solve by traditional optimization methods. The solution method presented in [6] allows the search for an optimum while considering simultaneously a large number of constraints. The GA method integrates all the constraints in the fitness function. The constraints are relaxed using a penalty, imposed for suboptimal solution. The main advantage which still makes GA a popular choice, is that the ratio of generated feasible solutions is rather high and the running time outperforms other approaches. This makes GA go beyond the simple heuristic methods. However, the promising results of the study considered only a simplified model of reality, with fixed costs and demands. As indicated in [16] an approach based on the Tabu Search method generates solutions comparable to the ones obtained using GA, considering small and medium size problems. Nevertheless, for complex problems, not only Tabu Search outperforms the Hybrid GA analyzed in [16] but also shows improvements with regard to Dobson's heuristic, "the best known heuristic method"[16].

It is remarkable the recent success of various researches in developing complex algorithms able to model the real-life production planning problems in careful detail. Nevertheless, it has not yet been found a formula not to face the compromise of having to balance an acceptable running time against a solution of a poorer quality. And it is very unlikely to build a general algorithm that can solve a production planning problem independent of its characteristics. Developed solution methods and previously obtained results represent a promising launching point in the field of production planning. Hence, the solution proposed in this master thesis tries to make use of the advantages of a metaheuristic method, based on the knowledge already achieved by other approaches.

2.3 Tabu Search applied on Production Planning problem

Whether small instances of a production planning problem can be solved in reasonable time to optimality by an exact method is still an open question. But for large, complex instances a method able to select and explore only parts of the search space is required. As the problem size increases, the search space's size is consequently growing exponentially. Being an NP hard problem, an exhaustive exploration of the search space cannot be performed. Therefore, one option would be to adopt a heuristic approach. The name finds its roots in the Greek *heuriskein: find, discover* and it refers to the technique of finding good (almost optimal) solutions for a certain optimization (mostly combinatorial) problem in reasonable time. However, optimality of a solution cannot be guaranteed and in many cases there is no evidence how close a discovered solution is to the (unknown) global optimum. The aim of such a method is to divide the space in regions and explore only the most promising ones. Therefore, a solution to the problem can be generated in a relatively reduced amount of time, since not all the candidate solutions will be tested.

A heuristic strategy will start from an initial solution, or a number of initial solutions, and will continue the search until no improvements can be brought to the current solution. Accordingly, the class of heuristic methods is divided in two subclasses: trajectory heuristics, starting with one initial solution, and population based heuristics, starting from a number of initial solutions. The method proposed in this thesis, Tabu Search (TS), is a trajectory based solution method, which has the advantage of an intensified search, in contrast to a population based heuristic (like e.g. GA), which aims at diversification, namely an exploration of larger regions of the search space. Briefly, population-based methods are better in identifying the promising areas of the search space. In contrast, trajectory-based methods are more suitable for exploring the promising areas. Additionally, population based methods face the risk of getting close to the global optima, but influenced by the knowledge stored in the population, to jump over it. On the other hand, a trajectory based heuristic can perform an intensified local search in these areas. Still population based methods are easier to parallelize and in case the search space is very large, a population-based method is suitable for exploring an increased number of subregions. Nevertheless, one more important aspect lays in the fact that a population based heuristic might be slower in converging to a very good solution, than a trajectory based one. Since individual solutions of a population based method influence each other, they may be entrapped in a local plateau if this is large enough.

However, both trajectory and population based heuristics perform a local search based on

a greedy mechanism that always tries to find the best next solution. Although heuristic methods are known to provide good results for uni-modal landscapes of the search space, one of the major drawbacks is that they cannot escape from a local optima in the case of a multi-modal problem. A uni-modal landscape ensures that once a local optimum is found, it represents also the global optimum. In contrast, in a multi-modal landscape local and global optimum are not necessarily equivalent. Therefore, the necessity of guiding the search arises, hence, a meta-heuristic algorithm. Namely, an enhanced heuristic algorithm that, by means of a set of adjustable parameters, can escape from local optima and continues the search in different regions of the search space in order to find solutions closer to the global optimum. A metaheuristic might represent a combination of diverse heuristic search strategies, able to perform both local and global search. The role of the meta-algorithm is to guide the embedded algorithm, having the aim of obtaining a more efficient or more robust procedure. The name combines the Greek prefix *"meta"="beyond"*, here in the sense of "higher level"and *"heuristic"="to find"*.

In what follows, the presented solution method is built on the basis of the Tabu Search meta-heuristic with adjusted parameters for production planning problems, using the discrete lot-sizing and scheduling model.

By means of memory lists, Tabu search is based on the interplay between intensification and diversification of the search in different regions of the search space. Intensification concerns the throughout exploitation of an interesting region, whilst diversification mainly aims to explore new regions of the search space. It follows the trajectory of a single solution, which is modified step by step until termination criteria are satisfied.

"The word tabu (or taboo) comes from Tongan, a language of Polynesia, where it was used by the aborigines of Tonga island to indicate things that cannot be touched because they are sacred."[9, 10] Webster's Dictionary offers the modern connotations of the word "a prohibition imposed by social custom as a protective measure"or something "banned as constituting a risk". The new meanings of the word accord with the underlying idea of TS, namely effective avoidance of a course which leads to entrapment in a local optima, by declaring it tabu. "The most important association with traditional usage, however, stems from the fact that tabus as normally conceived are transmitted by means of a social memory which is subject to modification over time. This creates the fundamental link to the meaning of "tabu" in tabu search. The forbidden elements of tabu search receive their status by reliance on an evolving memory, which allows this status to shift according to time and circumstance"[9, 10].

The key elements in TS are the tabu memory lists, which contain information collected during the search. Once a solution is visited, it is declared tabu for a certain number

of iterations, which prohibits the algorithm for visiting it again. In this manner, the TS strategy allows the search to escape local optima and avoid cycling to infinity.

Starting from a unique initial solution, a greedy local search is performed. By applying neighborhood operators, a number of so called candidate solutions are generated and the best one which is not tabu is selected to become the current solution of the next iteration. The term neighborhood operator denotes a method (function) which starting from the current solution provides a set of candidate solutions. By adapting the neighborhood operator one can perform intensification, by selecting a candidate solution which is slightly different than the current solution. Or, in contrast, diversification may be achieved, by selecting a candidate solution which significantly differs from the current one. Differences between solutions are to be defined with respect to the solution representation and customized according to the specifications of each problem. By monitoring the differences between the current solution and the generated solutions for a certain number of iterations, the algorithm automatically decides whether an intensification or a diversification phase should follow. This feature of a self-adapting algorithm by means of a dynamic neighborhood makes the Tabu search method very flexible and easily adjustable to the needs. Each solution has an associated neighborhood, generated by the neighborhood operator. At each step, a version of a descent method, scans the neighborhood of the current solution and finds the one which performs best. TS uses aggressive search in local neighborhood (best improvement local search). In contrast to an ordinary local search, TS accepts the new solution both whether it outperforms or underperforms the current solution. A pure local search algorithm stops once the current solution cannot be improved. Consequently, once a local optimum is found, the algorithm does not advance. Tabu search is already equipped to overcome this shortcoming by accepting solutions which, although do not improve the current one, allow the algorithm to advance, as the main goal of TS is effective avoidance of being stuck in local optima. Hence, TS augments the underlying principle of a greedy local search, by empowering the heuristic to escape from the attraction basin of local optima. However, this strategy leads to cycles. Therefore, a mechanism for the avoidance of cycles had to be developed, by prohibiting the repeated visiting of a solution.

In order to avoid cycling, the algorithm utilizes the memory of the search performed so far. The most recently visited solutions are set tabu, stored in the tabu list, and the algorithm is not permitted to revisit them for a certain number of iterations. As a consequence, the search will proceed from a local optimum, by creating a neighborhood of solutions and setting the previous local optimum as TABU. The algorithm is then forced to search for better solutions, except the local optimum. As it follows, the candidate solutions are not converging monotonously to the best found solution, as in the case of other algorithms,

but the optimum might be found in the middle of the searching process rather than in the end. As a consequence, the algorithm must keep track of the best solution visited so far.

All in all, the underlying concepts of Tabu search, namely the dynamic neighborhood operators and the usage of Tabu lists, give the algorithm flexibility and allow the exploration of the most interesting regions of the search space. Following the trajectory of the solution, a TS based algorithm is self-adaptive and can automatically adjust the search towards intensification or diversification. The memory lists allow the algorithm to proceed from a local optimum aiming at better solutions. Moreover, Tabu Search relies on a reduced number of parameters which must be adjusted and fine tuned. Therefore Tabu Search method represents a promising candidate for tackling complex problems like production planning.

Chapter 3

Problem Description

3.1 Background

This master dissertation thesis has been developed under the coordination of the *Company RISC Software GmbH* [3], which is a company of Johannes Kepler University Linz founded in 1987. Counting circa 40 employees, it is one of the most successful small-medium company in Upper Austria, with an annual turnover of EUR 2-3 Mio. One of the worldwide leading institute in symbolic computation, RISC melds together research, teaching and industrial development. The main areas of the RISC Software GmbH are Software development, Applied Research (algorithmic) and Transfer of Technology. In a nutshell, the company offers Advanced Software Solutions for Industrial Engineering and innovative approaches in:

* Analysis and simulation
* Modeling and optimization
* Planning and control
* Telematics

My project is focused on the branch regarding Production Planning for Company A that produces 3 Million units of product R per year by working in three shifts per day and eight hours per shift. Only during two shifts on Sunday the production is stalled. During the last years, the number of product types has strongly increased, whereas the sizes of the batches that have to be delivered to the customers are decreasing. To satisfy the customer needs with this larger number of product types and smaller orders, an intelligent planning

of the production is highly necessary. Currently, the company A supports 300 types of products and produces about 3000-3500 product units per shift. So far, the planning is done manually, with lots of experience and intuition. But the tougher conditions make an optimal planning more and more difficult and nearly impossible. Therefore, there is an increased need for a computer aided production planning.

An efficient planning of the production represents a considerable weight in the company's success. Hence its outmost importance lead to intensive research, which revealed different areas like lot sizing and scheduling, work force level, assignment of overtime and sequencing of production runs. A wide range of production processes are described by capacitated lot-sizing and scheduling models with complex constraints for multiple items, start-ups, and backlogging. Beside other techniques these optimization models can be solved by local search techniques for finding optimal or nearly optimal solutions. In order to achieve outstanding results, a local search technique might prove to be inefficient. Consequently, a metaheuristic method is the answer.

3.2 Problem Description

3.2.1 Project Topic

Production planning is the key word for the management of the production system and an important aspect of interest for researchers nowadays. The production process is described as containing two phases: a casting part and a cnc-mechanical part (computerized-numerical-control). Since the mechanical cnc part turns out to be the bottleneck of the production line, the aim is to model and optimize the production of this step with an individual production planning software.

A closer look at the cnc-production step reveals the following: There are 21 cnc-machines which work in parallel and are grouped in 3 different groups according to their machine type. The first group is of type BZ with 6 machines, the second one of the type PR with 3 machines, and the third group of type ST with 3 machines.

In order to work on a specific product type, the cnc-machines have to be prepared in two steps. Firstly, a suitable program has to be loaded and installed, so the machine knows how to process this specific product type. Secondly, a suitable clamping device has to be installed on the machine's robot arms, so they can grab the product. This two-step preparation of a machine is called the *setting* of the machine. Each time a different product type has to be produced on a machine, the setting has to be performed, which

takes up to four hours. During this period the machine stands still and cannot produce. Consequently, only a fixed and reduced number of settings are allowed per shift and per machine group. Basically there are different kinds of settings. Some product types are very similar and grouped together in product families. If there is a setting on a machine from one product type to another of the same product family, it is called a family setting, which takes approximately 90 minutes and the clamping devices need not to be changed. If a machine is prepared for a product type the very first time, it has to undergo special tests and the setting takes about four hours. Otherwise, a normal setting is done, which takes about three hours.

Moreover, the clamping devices are divided into two groups, type-specific ones and universal ones, and the number of the available clamping devices is limited. The type specific clamping devices are linked to a special product type and only work with that one. The universal clamping basically work with all product types, but are partitioned into subgroups by the size of the products.

Another important issue is that each cnc-machine has different production capacities which vary with respect to the produced product type.

Production is dictated by the demands, which are given per days for the considered time horizon of the planning. Additionally, demands further in the future are known as future demands and are given in accumulated numbers per week for the next four weeks. If the amount of the daily jobs is too little to fully load the machines, the free capacity is used to work on future demands.

Additional restrictions are imposed regarding the number of products of the same type that may be produced in parallel on different machines. Moreover, an upper bound for the production is established aimed to control the supra-production, in case of a reduced demand.

The goal is to schedule the entire process such that the production is maximal with minimum costs (settings costs, backlog costs) in a given amount of time. Consequently, the challenge is not only to find a feasible planning that fulfills the requests and obeys the restrictions, but also to find the optimal one.

3.2.2 Description

The aim of my project is to build an advanced, tuned meta-heuristic based on the Tabu Search method able to create a nearly optimal production plan that fulfills the requests and constraints of the Company A, for a simplified version of the above presented problem.

Company A has one group of machines that produce different types of products. In this respect, each machine has a production capacity, which differs with respect to the type of the product. The capacity matrix is given as cap[m,p] ≥ 0 of a machine m for product type p, 0 indicating that product p cannot be produced on machine m. Settings must be performed once a new product type is produced on a machine, but only regular settings (no family or first settings) are to be considered. However, their number still must not exceed a certain limit per shift. Also, clamping devices will not be considered.

The demands of the client are known as jobs in daily numbers considering a planning horizon that consists of five days with three shifts each. Additionally, the demands are known in advance for the next week. Therefore, in case the machines cannot work on their maximal capacity considering only the current request, future demands will be processed. This is called filling the machines. Also, in case the jobs cannot be processed in time, backlog costs have to be considered for the missing amount. On the other hand, in case production exceeds the demand in one shift, the products are stored in the inventory in order to fulfill a later demand. Initial storage and backlog numbers are provided by the company.

Considering the complex constraints of the problem, finding the optimal planning is translated into an NP-hard problem. Although attempts to solve the task manually have been made, increasing the number of products and varying the demands have as result an even higher complexity of the problem, making it extremely hard to be solved by hand. Hence, a specifically designed software must be created.

The model follows the strategy of the Discrete Lot Sizing Problem, which implies that the total time of the planning is divided into units, called micro-periods or shortly - periods. During one time period a machine should be set to produce units of only one item and it should work to its full capacity. However, since small periods are subject to modeling the simplest event that we are interested in - the setting of a machine, we decide that we allow at most one setting per period.

This assumption is valid in our case for one shift, but the setting itself does not take all the time of the shift and production may also happen in the same period when there was a setting. Hence, we go into modeling two events in a single period: first, the setting may occur only in the beginning of a period and second, production may happen immediately after the setting in the same time period. This results into a variation of the Discrete Lot Sizing and Scheduling problem (since two events are modeled in one period).

Expected Results:

- Based on a real-life problem from automotive supply industry, an optimization problem for production planning is modeled.
- For the solution of the combinatorial optimization problems a specifically tailored version of the Tabu Search is implemented, fine-tuned, and benchmarked.
- The results are compared to those of an existing mixed integer programming solution and a GRASP algorithm.

Steps:

- Sequential implementation of Tabu Search.
- Implementation and tuning of advanced metaheuristic techniques such as long-term memory strategies.
- Benchmarking (w.r.t. soultion methods, problem sizes, etc).

The above presented information was provided by Dr. Roman Stainko [7]

3.3 Mathematical Model

The input of the problem is defined by the following parameters:

Parameters:	
J	set of items (product types);
M	set of machines (21 machines of one type will be considered);
T	set of periods (describes the planning horizon - $5days*3shifts = 15shifts$);
cap_{mj}	capacity of machine m - represents the number of units of item j, that machine m can produce in one period;
$prod_{mj}$	{0,1} - indicates, whether machine m can produce item j (cap_{mj} can be used also);
I_{j0}	initial inventory(storage) of number of units of item j;
y_{mj0}	{0,1} - initial setup of machine m for producing item j at the beginning of the plan;
d_{jt}	number of units of item j demanded for period t;
s_{mj}	setting cost for setting machine m to produce item j (1 is the default value);
μ	maximal number of settings allowed in one shift
c_{jt}	backlog cost of not producing one unit of item j from the external demand d_{jt} in time period t (0.1 is the default value);
v_{mj}	ratio of setting time per working time in one period (3/8 is the default value);
b_j	buffer parameter describing how much overrun is allowed for item j on a single machine;
p_j	parallel factor of item j, indicating on how many machines parallel production of item j is possible in one period (3 is the default value);
F_j	future demands of units of item j;

Table 3.1: Production Planning Model's Parameters

3.3.1 Goal:

The goal is to create a planning that aims for limited amount of settings and minimal backlog costs. At a micro-level, the planning should indicate for each machine and each

shift the product that is produced and the quantity, given by the capacity of the machine. Additionally, since settings must be performed every time a new product is being produced on a machine, they should also be marked in the planning. At the macro-level, the output must contain information regarding the overall production in a certain shift. Namely, the unmet demand regarding the current requests, the unmet future demand and the quantity existing in the inventory after each shift.

The decision variables are given in the following table:

Decision variables:

x_{mjt}	{0,1} - indicates whether there is a setting performed in period t, on machine m in order to produce item j;
y_{mjt}	{0,1} - indicates the state of machine m in time period t, whether it is set to produce item j or not;
q_{mjt}	number of units of item j produced on machine m in period t;
u_{jt}	unmet external demand of item j in time period t, in number of units;
r_j	upper bound for unmet future demand of item j at the end, in number of units;
I_{jt}	inventory

However, the decision variables y_{mjt} or x_{mjt} are sufficient in providing all the requested information independently. Since they are a mirror one of another and all other decision variables can be computed using the input parameters and one of these decision variables, it is enough to consider only one. In what follows we will choose to represent our solution based only on the y_{mjt} and it will be called the major decision variable.

The inventory variable is not a major decision variable, since it is actually represented with a formula of parameters and other decision variables. Inventory of item j after time period t is defined as:

$$I_{jt} = I_{j(t-1)} + \sum_{m \in M} q_{mjt} - (d_{jt} - u_{jt}), \quad \forall j \in J, \quad \forall t \in T. \tag{3.1}$$

The objective function, which minimizes the setting costs, the backlog costs and maximizes the production for future demands, is defined in the following way.

Objective function:

$$\min \left\{ \omega_s \cdot \sum_{m \in M} \sum_{j \in J} \sum_{t \in T} s_{mj} \cdot x_{mjt} + \omega_u \cdot \sum_{j \in J} \sum_{t \in T} c_{jt} \cdot u_{jt} + \omega_f \cdot \sum_{j \in J} r_j + \omega_k \cdot \sum_{j \in J} k_j \right\}, \tag{3.2}$$

where ω_s, ω_u, ω_f, ω_k are weight coefficients. The objective function (3.2) aims at minimal cost of settings, minimal backlog cost, maximal production for future demand by minimizing the upper limits of unmet future demands, and minimal excessive production, subject to the following constraints.

Constraints:

$$q_{mjt} = cap_{mj} \cdot (y_{mjt} - v_{mj} \cdot x_{mjt}), \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.3}$$

$$y_{mjt} \leq prod_{mj}, \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.4}$$

$$\sum_{j \in J} y_{mjt} = 1, \quad \forall m \in M, \quad \forall t \in T \tag{3.5}$$

$$x_{mjt} \leq y_{mjt}, \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.6}$$

$$x_{mjt} \geq y_{mjt} - y_{mj(t-1)}, \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.7}$$

$$\sum_{j \in J} \sum_{m \in M} x_{mjt} \leq \mu, \quad \forall t \in T \tag{3.8}$$

$$\sum_{m \in M} y_{mjt} \leq p_j, \quad \forall j \in J, \quad \forall t \in T \tag{3.9}$$

$$r_j \geq F_j - I_{jT}, \quad \forall j \in J \tag{3.10}$$

$$I_{jT} \leq \max \{F_j + p_j \cdot b_j, I_{j0}\} + k_j, \quad \forall j \in J \tag{3.11}$$

$$q_{mjt} \geq 0, I_{jt} \geq 0, r_j \geq 0, k_j \geq 0, \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.12}$$

$$x_{mjt}, y_{mjt} \in \{0, 1\}, \quad \forall m \in M, \quad \forall j \in J, \quad \forall t \in T \tag{3.13}$$

$$0 \leq u_{jt} \leq d_{jt}, \quad \forall j \in J, \quad \forall t \in T \tag{3.14}$$

The constraint given by equation (3) limits the production in one shift with respect to the capacity of the machine for the produced item, taking into consideration also whether a setting has been performed.

Constraint (4) allows production if and only if machine m is able to produce product type j.

I_{jT} denotes the inventory in the last time period.

Equation (5) imposes that in each shift, on each machine only one product type may be produced, which accords to the DLS model.

(6) indicates that production is allowed only if the machine is set to produce the certain product type, while (7) indicates that a setting must be performed if a different product type is produced in consecutive shifts. Also the number of settings allowed per shift is limited to μ, by constraint (8).

(9) restricts the parallel production of an item on different machines to a certain number per shift. (10) imposes a restriction on the decision variable r_j, which is minimized in the objective function. T denotes the last shift of the planning horizon. (11) limits the amount allowed to be stored in the inventory at the end of the last shift with respect to the future demand. The slack variables k_j have the role to soften the constraint (11), but they are penalized in the objective function. These variables k_j are introduced for controlling the upper bound of the inventory I_{jT}, without making the plan infeasible.

Constraints (12), (13) and (14) allow only non-negative values for the decision variables and for demands.

Chapter 4

TABU Search

4.1 Solution Method - TABU Search

Hard optimization problems abound in the real world in all known domains. Exact methods are the answer for small instances of such problems, however medium and highly complex instances are tackled using meta-heuristic algorithms. "The abundance of difficult optimization problems encountered in practical settings such as telecommunications, logistics, financial planning, transportation, and production has motivated the development of powerful optimization techniques."[9]

Tabu Search (TS) took shape in 1986 in its basic form, founded on ideas of Fred Glover. The philosophy of TS is to exploit and boost a series of principles inherited from artificial intelligence, operation research and optimization. The method's underlying procedures are designed to dissolve boundaries of feasibility and local optimality, which were previously viewed as barriers to forbidden regions. Intensive research have brought important contributions to the evolution of TS, transforming this method into a successful one for many practical applications. "Tabu search is a meta-heuristic that guides a local heuristic search procedure to explore the solution space beyond local optimality. The local procedure is a search that uses an operation called *move* to define the neighborhood of any given solution. One of the main components of TS is its use of adaptive memory, which creates a more flexible search behavior. Memory-based strategies are therefore the hallmark of tabu search approaches."[9]

Adaptive memory and *responsive exploration* are a must in an intelligent search algorithm, according to Glover and Laguna. An illustrative analogy is mountain climbing, emphasizing that the climber must remember key elements from the path already traveled

(adaptive memory) and must take decisions along the way (responsive exploration).

4.1.1 TS Basic Strategies

The Tabu search meta-heuristic can be directly applied to verbally or symbolically specified decision or optimization problems, even without formulating a mathematical model. However it is useful to describe such models, for a better and complete understanding of the problem. In the case of an optimization problem, either minimization or maximization, one defines an objective function f(x) subject to x ϵ X, where X represents the set of feasible candidate solutions, given by the constraints. Highly complex constraints might be difficult to be translated into a mathematical formulation, in which case they will be coded as rules.

As a start, TS follows the mechanisms of a local neighborhood search, by proceeding iteratively from one solution to another, until some termination criteria are satisfied. For advancing from a certain solution x, an associated neighborhood for each x ϵ X must be defined, namely N(x) $\subset$ X, and each solution x' ϵ N(x) can be reached from x by an operation called a *move*. Nevertheless, a local neighborhood search only permits moves to neighbor solutions that improve the current solution and stops once such a neighbor solution cannot be found. Easily noticeable, such a solution is often a local optima and even the first one found. On the contrary, Tabu search makes use of guidance mechanisms that transcend a simple neighborhood search and escape local optima. One first step in achieving this target is to intelligently "determine an appropriate *candidate list strategy* for narrowing the examination of elements of N(x), in order to achieve an effective tradeoff between the quality of x' and the effort expended to find it."[9]. Therefore, TS bases its strategy on the memory for defining the modified neighborhood of x, denoted N^*(x). By maintaining a selective history of the search, both short-term and long term-memory, through their associated strategy have the effect of modifying the neighborhood of a solution x. Short-term memory imposes a tabu status on some solutions from N(x), and by excluding them one obtains the neighborhood N^*(x) as a subset of N(x). On the other hand, long-term memory may apply some strategies aimed to include new solutions in N(x). N^*(x) gives the list of candidate solutions, among which the best one is to be selected for the next iteration. From the above point of view, TS may be viewed as a dynamic neighborhood method, namely, the neighborhood of x is not static, but actually directly influenced by the history of the search. Hence, in the TS process a solution x may be visited more than once, but the associated neighborhood N^*(x) is very likely to be different every time. According to the history of recently visited solutions, a tabu-active tenure is associated to these solutions. Depending on the context, the number

of iterations for which the solutions are kept tabu may be variable. Generally, short tabu tenures allow the exploration of the solutions found in the very vicinity of a local optimum, whilst a long tenure is helpful in escaping the basin of attraction of a local optimum. As it will be explained later, this feature is the base of the *intensification* and *diversification* strategies.

Tabu activation rules can be overridden, by introducing aspiration criteria, thus removing a tabu classification otherwise applied to a move. An aspiration criterion applied even from the early versions of Tabu search and which was widely accepted over time is to remove a tabu status in case the generated solution is better than any visited before. A series of other aspiration criteria yielded good results. According to [9], "the appropriate use of such criteria can be very important for enabling a TS method to achieve its best performance levels."

For the basic version of Tabu search, involving only short-term memory based strategies, the above steps are repeated until the termination criteria are satisfied. These criteria can be given in a fixed number of iterations, which can also be translated as a time limit, until no further improvement was possible for a number of iterations, or a combinations of both. Surely, other criteria might be defined, depending on the context of the problem to solve.

Shortly, Tabu search starts by building a feasible solution, in the case of production planning, it represents a feasible plan. Then apply some neighborhood operators for a number of iterations to move from one solution to another, always making use of memory to avoid visiting the same solutions. TS uses aggressive search in local neighborhood, namely *best improvement local search*. In each step the best neighborhood non-tabu solution is accepted even if it is worse than the current solution. As this strategy leads to cycles, TS developed mechanisms for avoidance of cycles by prohibiting the repeated visiting of a solution. In order to achieve this, one utilizes the memory of the search performed so far, by declaring some moves forbidden (set Tabu). Once the termination criteria are satisfied, the searching process stops and the best solution visited during the whole search is returned. Note that TS does not show a convergence to the best found solution as in the case of other algorithm, but it may be found in any of the search phases.

Memory Structures

Memory is a key factor in Tabu search, therefore the method tries to fully exploit it. In this respect, TS makes reference to four principal dimensions: recency, frequency, quality and influence. *Recency* is a memory structure concerned with the recently visited solu-

tions and indicates which solutions have the tabu status. By means of recency memory, the algorithm can successfully escape from local optima and avoid cycling. A complement to recency based memory is the *frequency*. Together they are used for selecting preferred moves. The important characteristics of frequency based memory are emphasized in the Advanced Strategies section. *Quality* is a dimension that "refers to the ability to differentiate the merit of solutions visited during the search. (...) memory can be used to identify elements that are common to good solutions or to paths that lead to such solutions."[9] Consequently, based on the quality dimension of the memory, one decides which action are favored in order to generate promising solutions and which actions are to be penalized for discouraging the visiting of poor solutions. The flexibility of these memory structures allow Tabu search to operate successfully even in a multi-objective environment. The search direction is guided by determining how good a solution is, using more than one function. Finally, *influence*, the forth memory dimension considers the impact of the choices made during the search process, not only from the point of view of the quality of the solutions, but also regarding the structure.

With respect to the items stored in the memory lists, the memory is classified as *explicit* and *attributive*. Explicit memory records complete solutions, most often elite solutions encountered during the search or highly attractive but unexplored neighbors of these solutions. Based on the memorized solutions, TS expands the local search towards the most promising regions. However, an explicit memory might be memory consuming, depending on the size of the solutions. As a consequence, it is designed to store only a reduced number of solutions. For the entire history of the search, attributive memory is the key solution. Namely, storing only information about the solution attributes that are changed in the process of moving from one solution to another. As an example, "in production scheduling, the index of jobs may be used as attributes to inhibit or encourage the method to follow certain search directions."[9] Hence, the attributive memory has merely guiding purposes, by inducing penalties or inducements to forbid or favor preferred moves.

All in all, memory usage is an important characteristic of the Tabu search method and also the feature that distinguishes its strategies from the other methods' optimization techniques.

4.1.2 TS Advanced Strategies

In some applications, short term memory techniques are sufficient for high quality results. Nevertheless, TS becomes significantly stronger by applying long-term memory

based strategies. The widely adopted approach is to run the short-term memory based mechanisms for a number of iterations, and once they become inefficient to give power to the algorithm by extending the neighborhood N(x), of the current solution x, using information gathered during the whole search process. The modified neighborhood provides new solutions consisting of selected elite local optima encountered at various points in the process, or solutions built from components of the elite solutions. A surprising remark is that long-term memory and the associated strategies yield very high quality results in a modest length of time. Therefore, it is required only a reduced number of iterations until the benefits of these strategies become visible. Long-term memory "can allow solution efforts to be terminated somewhat earlier than otherwise possible, due to finding very high quality solutions within an economical time span"[9]. In a nutshell, the results provided by the Tabu search method are rapidly enhanced by applying long-term memory strategies, in addition to short-term memory ones.

Frequency-Based Approach

One form of long-term memory is keeping track of the frequency of the attributes in the visited solutions. As indicated by Glover and Laguna in [9], frequency based memory provides information that complements the one provided by recency-based memory, thus broadening the foundation for selecting preferred moves. Frequencies are represented as ratios. The numerators are expressed as counts that indicate *transition measure* - the number of iterations an attribute changes by being added or dropped from the visited solutions; and *residence measure* - the number of iterations where an attribute belongs to the solutions visited on a particular trajectory. The denominators may be one of the following counts: (1) total number of iterations, (2) the sum (or average) of the numerators or (3) the maximal value of the numerators. According to the type of the numerators, one obtains *transition frequency* keeping track of how often attributes change, or *residence frequency* that keeps track of how often attributes are members of the generated solutions. Once defined, the frequencies can be used to influence the selection of moves in the solution generation process. In this respect, "a high residence frequency may indicate that an attribute is highly attractive if the domain consists of high quality solutions, or may indicate the opposite, if the domain consist of low quality solutions."[9] In contrast, in a mixed domain, a high residence frequency may point to an entrenched attribute that causes the search space to be restricted. In this case, a move that drops this attribute in order to allow diversity is preferred. A reversed strategy should be applied in case of a low residence frequency. Distinctively, a high transition frequency may indicate that the corresponding attribute represents a *crack filler*, namely an attribute that shifts in and out from a solution to perform a fine-tuning function. These are highly attractive

attributes, from the point of view of their cost, but it may be the case that they do not belong to the optimal solution. In this situation they enter and leave the solutions alluring the search away from the optimal region. However, crack fillers are determined not only by cost and quality, but also, importantly, by structure, as they must connect to other attributes for building a solution. Some of the crack fillers may be part of the optimal solution, but they are difficult to be identified in the early stages of the search, but merely in the late ones, once some other attributes are fixed in their positions.

Although frequency measures are used mostly by long-term memory strategies, they can also have an influence regarding the tabu status of an attribute. In this respect, attributes showing a high frequency measure, combined with a recency measure may be assigned a tabu status if consecutive solutions containing the attribute end with the current solution, as explained in [9]. However, frequency-based memory finds its most productive use integrated in the long-term memory strategies. Its role is to employ incentives or penalties to influence the selection of the move to be applied at each step. Similarly to the tabu activation rules, restrictions are imposed or incentives are used to modify the rules by which certain moves are qualified as attractive or unattractive. Moreover, frequency measures may be used in some cases to override a tabu status of an attribute. For instance, high residence frequency of an attribute indicates the need of assignment of a strong incentive to favor a swap of that attribute in order to allow the search to advance. This may prove to be very useful in case the attribute is tabu and this tenure was assigned for a considerable number of iterations, thus restricting the search.

In the production planning problem an attribute may be defined as the triple (machineID, shift, productiID). Under these conditions, a transition measure stands for the number of iterations such an attribute has been added/dropped from the visited solutions, while the residence measure indicates the number of iterations an attribute belongs to the solutions having been generated.

Frequency-based memory represents one type of long-term memory and it is extremely beneficial as the underlaying principle of the advanced tabu search strategies presented next.

Intensification

Intensification strategies have the role of modifying the choice rules to encourage the visiting of solutions that combine features historically found to be good. Their aim is to return to attractive regions and search them more thoroughly. The simple form of the intensification TS algorithm is:

1. Apply TS short term memory
2. Apply an *elite selection strategy*
3. while (nrIt $\leq$ limit) and (list_of_elite_solutions non-empty)
    - Choose one of the elite solutions
    - Resume short term memory TS from chosen solution
    - Add new elite solutions to the list when applicable

While applying the short-term memory based strategies for a number of iterations, one collects also medium and long-term memory information, like the visited elite solutions and their attributes. The question is how does one decide which are the elite solutions. In order to answer, there are several variants for the elite selection, some of them having proved to be very successful.

The first strategy imposes a diversification measure, namely record high quality solutions that differ from each other to a certain degree. Then erase the short-term memory and resume the search from the best recorded solution. Two solutions are said to be different from each other if a large number of moves is necessary to transform one solution to another. Another way of defining it, is by the number of attributes in which the solutions differ. From this point of view, one can conclude that intensification and diversification work together, as it will be explained later. A second elite selection strategy makes use of a sequential list of fixed length containing the best solutions found so far. The principle is to add a solution to the end of the list only if it better than the best previously visited solution. Then resume short-term TS always from the last solution in the list and remove it. In order to avoid following the same trajectory, one uses the associated short-term memory of the elite solution and forbids to apply the same first move, so that a different path will be launched.

A third strategy keeps track of the high quality unvisited neighbors of the solutions visited so far, selects an elite set of these neighbors and resumes the search from these solutions. However, this strategy has been little examined, in contrast to the previously presented two strategies that yielded very good results and attracted a high interest. Finally, another successful approach is *intensification by decomposition*, which imposes restrictions of some components, in order to focus on the other parts of the structure. These restrictions decompose the solution in the fixed parts and the components that are changing their values to improve the solution, thus the name of the strategy. One way of determining the set of fixed attributes is by intersecting the elite solutions. As an example, let us consider a permutation problem, as in the case of scheduling or routing,

where solutions may be represented as sequences. The first step is to find subchains of elite solutions and separate them in sets in the following way. A subchain will be assigned to a set if and only if it can be linked to another subchain from the set. Then perform intensification in each subchain, without altering the end-nodes, such that linking inside the sets is still possible. Finally, the best new sets of subchains will be reassembled to create new solutions. "These ideas are lately finding favor in other procedures, and may provide a bridge for interesting components of tabu search with components of other methodologies. "[9]

Intensification is a medium-term memory based strategy and it is applied periodically as the short-term memory based solution results cannot be improved by standard strategies. Nevertheless, one should distinguish between *intensification* and *exploitation*. While the first one uses memory information, the latter follows a recipe (usually memoryless), until it fails to be effective.

Diversification

Diversification strategies are designed to drive the search into new regions, once the short-term memory strategies become ineffective. Also, the quality of a diversification strategy is that it prevents the algorithm from cycling, enhancing it with robustness and vigor. In many different approaches like Genetic Algorithms, Simulated Annealing or GRASP (Greedy Randomized Adaptive Search Procedures) randomization is involved, but in TS diversification is achieved by means of the long term-memory. Based on the frequency measures, the strategy aims to bring infrequently used attributes into the candidate solutions in order to explore new regions of the space. One way is to introduce these attributes into the current solution and continue the search or to periodically assemble subsets of these attributes into candidate solutions and then partially or fully restart the searching process from the newly generated solution. The diversification step is particularly helpful for crossing barriers in the search space, when no further improvement of the solution is possible. Two widely used approaches to perform diversification are modifying the choice rules for building the next candidate solution or to restart the search procedure from a different point.

Modifying Choice Rules: Again using the frequency measures of the attributes, diversity can be achieved by assigning penalties or incentives to the moves in order to forbid or favor the inclusion of certain attributes in the candidate solution. In order to exemplify this strategy more thoroughly, one can analyze the following example. Consider as input a graph. The goal is to partition the nodes of the graph in two *equal* sets, such that the

sum of the weights of the edges that join nodes in one set to nodes in the other set is minimized, as presented in [9]. The TS method applied uses two neighborhood operators, that *swap* two nodes from different sets or *partial swap* that transfers one node from one set to another. Clearly, a full swap will preserve feasibility, while a partial swap will not. However, periodically, in order to generate diversity, one approach is to disallow the use of non-improving full swaps, for a certain number of iterations. Therefore, only partial swaps or improving full swaps are permitted. The consequence is that the solution will become infeasible, and feasibility must be restored afterwards using specially tailored techniques. "Implemented appropriately, this strategy has the effect of intelligently perturbing the current solution, while escaping from a local optimum, to an extent that the search is directed to a region that is different than the one being currently explored. The implementation of this strategy as applied to experimental problems has resulted in significant improvements in problem solving efficacy."[9]

The preference of partial swaps against full swaps is controlled using the following penalty function:

MoveValue' = MoveValue + d*Penalty,

where Penalty value is a function of frequency measures and d is an adjustable diversification parameter. Large values of d correspond to a high degree of diversification. Values of Penalty are computed such that moves that incorporate attributes with low frequency are preferred. Also, negative penalties, called inducements, may be used.

Restarting: A second mechanism for achieving diversity is to restart the search from newly created solutions. For this, one uses frequency information to create a new initial solution by incorporating attributes with a low frequency from the already visited solutions. The search is then resumed from the new candidate solutions. In contrast to Intensification, which uses the elite solutions found during the search, diversity is a strategy based on all solutions visited so far, as the aim is to explore regions which where not visited before.

Diversification vs Randomness: As in the case of intensification, *diversification* and *exploration* should not be taken as tantamount terms. Diversification is a long-term memory based strategy, while exploitation makes use of a series of random changes. Including the random element is a widespread mechanism for achieving diversification used by various optimization methods. However, in Tabu search a diversification step is performed by purposeful moves, guided by frequency measures. Even if perturbations are applied and a random element is sometimes involved, it is confined by limits computed

using memory information. The strong principle of TS is to apply randomization not as a "blind mechanism", but to use memory information that confine the random element. "In short, the situations where randomization finds a place are very highly structured. From this point of view God may play with dice, but beyond any question the dice are *loaded*" [9].

Strategic Oscillation

Strategic oscillation is in close connection with the origins of tabu search and it is based on the interplay between intensification and diversification, by making use of the intermediate and long-term memory. "Strategic oscillation operates by orienting moves in relation to a critical level, as identified by a stage of construction or a chosen interval of functional values. Such a critical level or *oscillation boundary* often represents a point where the method would normally stop." [9] However, instead, TS continues the search by modifying the move rules such that the selected moves allow the search to cross the boundary and explore both regions delimited by the critical level. The approach then proceeds for a specified depth beyond the oscillation boundary and turns around. Then again approaches and crosses the oscillation boundary from the opposite direction to a new turning point. This imprints an oscillatory behavior on the trajectory, thus giving the name of the strategy, as depicted in Figure 4.1:

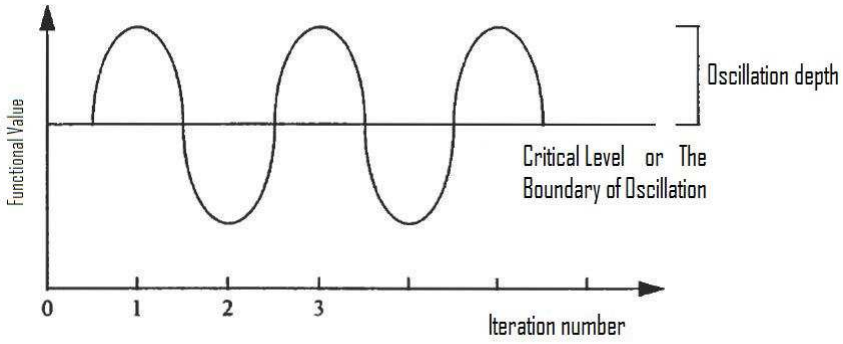


Figure 4.1: Strategic oscillation

The control over the oscillatory behavior is maintained by modified evaluation and move rules, depending on the currently explored region and direction of the search. It is again a must to use standard TS mechanisms to avoid following a prior trajectory, aided by recency-based or frequency-based memory.

As a simple example synthesizing the strategic oscillations method, one may consider the Min k-Tree Problem. Namely, given a graph and the weights associated to its edges, one has to determine a subtree of k edges such that the sum of weights of its edges is minimal. Critical level will represent the feasibility boundary, hence a tree with k edges. The first step is to generate a solution by progressive insertion of edges beyond the critical level defined by k and explore the region with more than k edges. Then apply a rule to delete edges, again cross the oscillation boundary and go into the region with less than k edges. All feasible solutions lie on the oscillation boundary, therefore it is required to spend additional time on the critical level and very close to it to explore the neighborhood of these solutions more thoroughly, by performing swap moves. Additionally, the method is very flexible and adaptable to each problem's characteristics. In this respect, the search can be guided to cross the boundary on different depths on different sides. Or even the extreme case, remain on one single side, called "zero depth" and explore only one of the regions delimited by the critical level. Since infeasibility is allowed, one has to compute the degrees of feasibility and infeasibility as a function of values assigned to the constraints. "In this case, controlling the search by bounding this function can be viewed as manipulating a parameterization of the selected constraint set."[9]

Macro level decisions: Before applying the strategic oscillation method, one has to take some decisions regarding the key factors of the mechanism at a high, overall level. The steps are:

1. Select an oscillation guidance function

   This refers to the element being controlled like tabu restrictions, infeasibility penalties or objective function. Then the function is a measure of this element.

2. Choose a target level for the function

   The aim of the target level is to guide the searching process. Note that it is defined as a *target level*, rather than a *target value* as it may be conceptualized as a spanning interval including the critical level. Few examples of target levels in different problems are: in the graph problem the target level can be established as a certain graph structure ; or in the problem of assigning jobs, for instance, a complete assignment.

3. Choose a pattern for the oscillation

   This last step is related to the depth by which the search goes beyond the target level in each region and how depth varies over time. Is also concerned with the steps taken in approaching the target.

Micro level decisions: Involve variability of a particular local form. The three micro decisions are:

1. Choose the target rate of change - to move away or toward the target level

   The way of approaching the target level can range from mildly to moderately aggressive, hence the trajectory may show a mild/steep descent behavior. TS uses aggressive seeking, or steepest descent in the Intensification phases.

2. Choose the target band of change

   Sets the boundaries of deviation from the target rate.

3. Identify aspiration criteria to override target restrictions Target rate and band explore the current neighborhood, but impose restrictions. Hence an aspiration criteria must be invoked to allow a move if it generates a solution better than the best one encountered at the current oscillation level.

Other strategies embedded in strategic oscillations:

Intensification is achieved by modify choice rules to include or fix high quality attributes in the solution and to spend more time in the promising areas, for instance on certain oscillation levels or on the critical level.

Diversification can be represented as an extreme type of oscillation, analogous to a restart method. Hence, the strategy will use only construction procedures and explore both regions, but once it reaches the turning point, instead of turning, it restarts from the same initial level, with new choice rules.

Perturbation of the solution might induce an oscillation behavior, and although a critical level must not necessarily be defined, this method "seek goals similar to those pursued by strategic oscillation."[9]

Proximate Optimality Principle (POP) encourages the hovering about the target level and other oscillation levels. According to this principle, good solutions can be found close to good solutions at an adjacent level, namely only a reduced number of steps is required to reach good solution at one level from those at another. This notion is the underlying principle of Path Relinking, strategy that will be presented in the next section.

Strategic oscillation in Production Planning applications: It is often the case that for scheduling problems the oscillation boundary is chosen to be the feasibility boundary, considering that all machines are working at full capacity. One region will contain overloaded machines, the other will contain idle machines. In the PP problem tackled in this master thesis, the region containing overloaded machines will be unexplored - "zero depth", and the boundary is approached in oscillatory behavior from one side, by deleting sequences of lots, reach the turning point and then insert new lots. Additional time will be spent on the boundary, by using swap moves - exchange lots or replace items with new ones.

Path Relinking

Path relinking represents an advanced strategy which is not necessarily characteristic to the tabu search method, however it is often applied as a post-search mechanism for the

results' improvement. Based on POP principle, it generates new solutions by exploring trajectories between elite solutions found by short-term memory TS. The approach starts by selecting an initiating solution and one or more guiding solution, then step by step includes attributes of the guiding solutions in the current one. In the case of more guiding solutions, weights are assigned to attributes in order to favor or disfavor their inclusion. Hence, larger weights correspond to attributes that belong to a greater number of elite/guiding solutions. High weights may be assigned to attributes which although are not contained in such solutions, are very similar to those attributes or are promising to yield high quality solutions. During the process of path relinking, initiating solutions and guiding solutions may be alternated, if the current corresponding solution of an initiating one becomes highly attractive. Therefore they are all referred to as *reference solutions*.

Similar to the strategic oscillation method, path relinking can include other advanced strategies. As it follows, intensification is achieved by linking similar solutions (elite solutions) ; or a diversification step is performed by linking dissimilar solutions or solutions from different regions. Although Tabu search is a deterministic method, in the path relinking step one may use deterministic or probabilistic strategies to favor attributes that lead to promising regions, in addition to the associated weights. In the case of one initiating solution and one guiding solution, the path usually stops once the guiding solution has been reached. Nevertheless, the path may be continued after reaching the guiding solutions by penalizing the inclusion of the dropped attributes along the trajectory and favoring the inclusion of attributes from other guiding solutions. In many path relinking versions, feasibility is allowed for a certain degree such that wider regions are explored during the search. However, it has to be controlled and restored during the process. Moreover, aspiration criteria is applied to permit visiting of high quality solutions which are otherwise restricted.

Incorporating Alternative Neighborhoods: In the process of path relinking different neighborhood operators than those used by TS for generating the reference solutions are needed. The new neighborhood operators should be focused on replacing attributes from the initiating solution with attributes contained in the guiding solutions. There are two widely used approaches to achieve this:

1. Constructive neighborhood

   In this case, the initial solution determines a partial solution, by selecting only a subset of its attributes. In the extreme, the initiating solution is the null solution. Then, progressively add attributes from the guiding solutions to the partially constructed initial solution.

2. Destructive neighborhood

   Another approach is to start with an initial solution overloaded with attributes from guiding solutions. Again an extreme strategy would be to include all attributes from guiding solutions and then progressively drop attributes until a feasibility is reached.

In contrast to transitive approaches, by applying this neighborhood operators only one solution is generated in the process of path relinking. Hence there are less chances to visit other high quality solutions during the search. The solution method is to combine this approach with the transition neighborhood, namely swapping attributes, to reach additional solutions. As in the classical path relinking strategy, feasibility restoring neighborhoods methods must be applied.

To conclude, long term memory strategies are designed to help the short term memory based Tabu Search to improve the quality of the generated solutions. Not only the advantage lies in a higher quality results, but the benefits of the advanced strategies become visible in a rather short time.

Chapter 5

Implementation Details

The input parameters of the Production Planning problem together with the decision variables involved in the solution require a relatively large amount of memory in order to be stored. Consequently, the representation of the parameters might not be an exact mirror of the one described by the mathematical model. For instance, the binary variables X and Y are transformed from a 3D matrix with {0,1} values into a 2D matrix with integer values, as depicted in what follows.

5.1 Representation of the Solution and Parameters

5.1.1 Solution Representation:

In the PP problem, a solution is given by a feasible planning. Therefore our aim is to choose a solution representation that will not only make the processing of the solution easier, but will also include some of the restrictions of the problem. In this respect, a solution is represented in the current thesis by the production planning Y, as a two-dimensional matrix, with M rows and T columns. M represents the number of available machines, while T indicates the total number of shifts of the planning time horizon. Each entry Y[m,t] indicates the production of machine m in shift t and it contains an integer j, which gives the identification index of the product type being produced in that shift. Thus, one of the DLSP constraints is achieved by using this representation, as in one shift, on a particular machine, only one type of products can be produced. Additionally, the production matrix Y offers indications about the settings. Hence, a setting is performed if the current item produced on the current machine is different than the previous item. In other words, whether two consecutive cells of the same row contain different entries

Y[m,t] $\neq$ Y[m,t+1], the later one models both the setting and the production. Namely on machine m in shift t+1 a setting occurs for the item indicated by the value Y[m, t+1] and then production starts.

The following figure 5.1 illustrates a representation of the solution for a production planning problem with M=6 machines, T=9 shifts and J=12 products:

	Shift1	Shift2	Shift3	Shift4	Shift5	Shift6	Shift7	Shift8	Shift9
M1	P1	P1	P5	P4	P3	P3	P3	P6	P6
M2	P6	P7	P7	P2	P2	P2	P2	P1	P1
M3	P5	P5	P3	P3	P11	P4	P9	P9	P9
M4	P2	P2	P1	P1	P8	P8	P9	P7	P7
M5	P10	P10	P9	P3	P3	P12	P12	P12	P11
M6	P7	P7	P7	P6	P11	P11	P9	P8	P8

Figure 5.1: Solution representation - production plan

An alternative to the Y representation of the production plan is to use the planning of the settings X, since Y and X are equivalent one to another under the conditions of the DLSP problem. X is represented as an array of M arrays of variable length, where again M indicates the number of machines and each entry contains the tuple (productID, starting shift). The starting shift value indicates when a setting occurs and the duration of production for that particular item is computed with respect to the starting shift of production of the next item on the current machine. The equivalent representation of the production plan Y using the setting plan X is given bellow in figure 5.2:

Setting, Shift					
M1	P1, 1	P5, 3	P4, 4	P3, 5	P6, 8
M2	P6, 1	P7, 2	P2, 4	P1, 8	
M3	P5, 1	P3, 3	P11, 5	P4, 6	P9, 7
M4	P2, 1	P1, 3	P8, 5	P9, 7	P7, 8
M5	P10, 1	P9, 3	P3, 4	P12, 6	P11, 9
M6	P7, 1	P6, 4	P11, 5	P9, 7	P8, 8

Figure 5.2: Solution representation - settings plan

In this thesis, the Y matrix of the production plan is chosen as the representation of a solution to the PP problem. Additionally the solution information contains the values for M and T, the number of machines and the number of shifts respectively, and the objective value associated to the plan given by the matrix Y.

5.1.2 Parameters Representation:

The other parameters required for solving a PP problem are:

- The values J, M, T stored as integers are the number of products, machines and shifts respectively.
- int** capacity; is the capacity matrix capacity[M][J], indicating the capacity of machine m (rows) for product j (columns); the entry will be 0 if m cannot produce j.
- int** demand; the matrix of demands[J][T], shows the number of units of product j (rows) that are demanded in the shift t (columns)
- int* future; a one-dimensional array future[J] gives the future demand for each item j
- int* buffer; buffer[J] signifies the overrun allowed for each item j
- int* init; init[M] is the array containing the initial settings for each machine
- double* stock; stock[J] contains the initial inventory for each product
- double** u; the matrix of unmet demands [J][T] is one of the decision variables indicating the negative inventory for product j (rows) in shift t (columns)
- double** I; inventory[J][T] matrix indicates the positive inventory for product j (rows) in shift t (columns)
- double** q; another decision variable is the matrix quantity[J][T] and it indicates the quantity of an item j (rows) produced in the shift t (columns), on all machines
- double* r; r[J] represents the upper bound for the unmet future demand for each product j
- int* nr_set; nr_set[T] keeps track of the number of settings performed in a shift t, on all machines

- int p_; indicates the maximal number of parallel productions allowed for an item in the same shift
- int sigma; gives the maximal number of settings allowed per shift
- double* sumProd; sumProd[T] each entry indicates the total number of units produced in each shift t
- double* sumInv; sumInv[T] each element of the array contains the total number of units in inventory in each shift t
- double* sumUnmet; sumUnmet[T] represents the total number of unmet demands in each shift t
- int** Y; the production matrix Y[M][T] indicates which product is produced on machine m(rows) in shift t(columns); no machine can be idle, therefore, each entry takes an integer value between 1 and J indicating the index of the item being produced
- int** X; the settings matrix X[M][T] contains the index of the item if a setting is performed on machine m (rows) in shift t (columns), otherwise 0.
- double s_, c_, v_; represent the setting cost s_, the cost c_ for not meeting a demand, and the cost v_ for not meeting a future demand; All costs are computed per unit for each product.
- double omega_s, omega_u, omega_f, omega_k; represent the weight coefficients used in computing the objective value. Therefore omega_s is the settings coefficient, omega_u the coefficient for unmet current demand, omega_f the coefficient for unmet future demand and, omega_k a coefficient introduced to penalize the supraproduction in the objective function. [The constraint that limits the production of the items with respect to an upper value considering the future demand is treated as a soft constraint and is penalized in the objective function.]
- double objective; represents the objective value associated to the production plan .

However, storing these parameters is memory consuming, therefore they are read from files and stored in an object accessible for the algorithm in order to make the necessary computations, but they are not contained in all the solutions visited during the search.

5.2 Initial Solution Construction

The construction of the initial solution emphasizes the very large demands. In order to be able to fulfill them, one has to start to produce the requested items in advance. For this, one finds an available machine that can fulfill the demand and computes the number of shifts necessary for producing the requested quantity. Then assign the job for that certain machine, such that the demand will be fulfilled in time. Production will end in the shift when the demand is due to. The shifts earlier than the starting of the production can be used to produce other items and fulfill other demands. In case no machine is free to produce the requested item, some products will be moved on different machines, in the most favorable configuration. At each step feasibility is verified, consequently the algorithm ensures the construction of an initial feasible solution.

5.2.1 Construction Algorithm

The algorithm for constructing the initial solution is sketched as:

1. List of demands sorted in descending order with respect to the requested quantity
2. For each item p in the list of demands
   (a) Find free machine
   Else
   (b) Find busy machine
   For all products that have to be moved, find another machine

First step is to sort the demands in an one-dimension array, descending with respect to the quantity that is demanded. Then find the machine *mfree* that can produce the item p and is free to produce p for the necessary number of shifts. In order to make the minimal number of settings necessary for the configuration being built, one first searches for a machine *mfree* which does not require a setting. Either an initial setting is already made, or the machine was already producing the same item in the previous shift. If no such a machine can be found then search for a free machine, even if a setting is required. If the solution obtained by inserting p is valid, insert product p on machine *mfree* for the necessary number of shifts.

If no free machine has been found for product p, the next step is to find the machine *mbusy* that can produce p and the solution obtained by inserting p is valid. Then insert p

on machine *mbusy* and additionally create a list of products *prods* that have to be moved on different machines. Before processing the next demand in the sorted list of demands, one proceeds by finding another machine, free or busy, for all the products in *prods*. In case no machine is found for some of the items in the list *prods*, a list of rejected demands is created and they will be assigned to earlier shifts in the next step of the construction process.

For inserting a product, one has to process one element of the array of sorted demands. This element is a triple of the form (prodID, shiftNr, quan), indicating that the due time for producing a quantity *quan* of item *prodID* is shift *shiftNr*. Firstly, one analyzes each machine that can produce the item *prodID* and computes the number of necessary shifts to fulfil the demand *quan* according to the capacity of the machine. This yields the value *reqSh*, the number of required shifts. If $reqSh \leq shiftNr$, then insert *prodID* on the machine, *mfree* or *mbusy*, in the shifts starting from $shiftNr - reqSh$ until *shiftNr*. In case $reqSh > shiftNr$, insert *prodID* on the current machine, and the remaining quantity for fulfilling the demand is inserted in the list of sorted demands, in order to be produced on another machine.

When no machine can be found for fulfilling a demand, this demand is added to a list of rejected demands *rejDemands*. Next, after all the demands in the sorted list have been processed, for each item in the *rejDemands* the algorithm tries to find a machine that can produce any part of that demand in any earlier shift. To achieve this, the partial solution is parsed and all free machines which can produce item *prodID* are analyzed. If there is a free shift and the shift $\leq shiftNr$ then verify the preservation of feasibility. If by inserting *prodID* a valid solution is obtained, then insert the item and remove the quantity from the list *rejDemands*. If no machine can produce the rejected demand, it will be removed from the list *rejDemands* and the backlog costs will be computed. This procedure is repeated until the list *rejDemands* is empty and its aim is to reduce the unmet demand and also to fill the idle machines. When due to feasibility reasons the machines could not be filled with the current demand, the following step is to take the future demand into account. Therefore, if there exist idle machines, the algorithm proceeds by creating the list of future demands *fdemands*. Next, for each item in the *fdemands* one finds a suitable machine which can produce the particular item and also by inserting the production of the item in the idle shift, the obtained solution remains valid. In processing the future demands, in contrast to the current demand, there is no constraint regarding the due shift, as all items in the *fdemands* list are treated as having the deadline the last shift of the planning horizon.

Finally, when after processing the future demands there still exist idle machines, the idle shifts will be occupied by extending the lots from the previous shifts. In order to

remain feasible, one has to soften the constraint regarding the maximal overrun of the production. Therefore, supra-production will be allowed, but harshly punished in the objective function. As a consequence, for the incipient phases of the search algorithm, the generated solutions may produce a quantity larger than requested for some products, but as the search evolves, these items will be eliminated in favor of those for which an unmet demand has been registered. Hence, the aim is that the final solution will contain a plan for which the production overrun is minimal or even zero.

The advantages of the presented algorithm are that it is focused to reduce both the number of settings and the number of backlog costs. Hence:

- It starts by processing full production lots for each item, rather than each production per shift individually, thus reducing the number of settings;
- Products which could not have been assigned in the due shifts, are assigned in earlier shifts, by this reducing the number of unmet demands;
- Future demands are considered in order to fill the idle machines;

On the other hand, some disadvantages may also be noticed. Once a product is assigned for a shift, it is not fixed, therefore, one has to reevaluate the solution after each change. In this respect, when no free machine is found, products which have been assigned to the chosen machine have to be moved again. Nevertheless, the algorithm is very fast in providing an initial solution, which is later improved by the meta-heuristic.

5.2.2 Results

For testing the algorithm we have considered six problems: three more complex problems and other three smaller instances of production planning problems. The results for each of the testing problems are given below:

Results						
Problem	Objective Value	Setup Costs	Backlog Costs	Future Costs	Inventory Violation	Computational time
Big 1	245.29	84	150.1	0.6842	10.512	$<$ 1 millisec
Big 2	637.263	80	207.04	0.5375	349.68	$<$ 1 millisec
Big 3	1081.31	80	576.62	0.5668	424.12	$<$ 1 millisec
Toy 1	586.58	27	558.65	0.9383	0	$<$ 1 millisec
Toy 2	173.88	26	146.97	0.91361	0	$<$ 1 millisec
Toy 3	309.05	29	279.62	0.9283	0	$<$ 1 millisec

To conclude, the underlying idea of the algorithm for construction the initial solution is that production for large demands has to be started early in order to avoid backlog costs. The algorithm aims for a balance between the total number of settings and the backlog costs for the unmet demands. Consequently, large production lots are preferred to be processed as a whole, thus decreasing the number of settings. Additionally small early demands lose priority in front of larger demands, for the continuity of the lots. Nevertheless, backlog costs are reduced by assigning small production lots to the idle machines.

5.3 TS Short-Term Memory

Once an initial solution has been constructed, the aim of the meta-heuristic algorithm is to improve it next to optimality. Having this aim, two types of moves are defined: the first one swaps the positions of the elements contained in the solution, and the second one replaces one element with a new one. From the point of view of the optimization purposes, there are three types of costs which are minimized by the objective function, namely the set-up costs, costs for unmet current demand and costs for unmet future demand. Since the current and future demand may be treated in the same way, two neighborhood operators are defined: one designed to reduce the number of settings and a second one which aims at reducing the costs for unmet demands. Hence, the neighborhood operators' role is to generate new candidate solutions such that the production and backlog costs are minimized. Applying the operators for a predefined number of steps, a set of solutions is generated in each iteration. Among them, the non-tabu solution having the minimal objective value is selected to be the starting solution for the next iteration. As in the classical version of TS, the tabu list and the best encountered solution are permanently updated and the above presented steps are repeated until the termination criteria are satisfied. In what follows, both the neighborhood operators and the Tabu

Search algorithm will be explained in more details.

5.3.1 Neighborhood operators

Reduce Settings Operator:

The first neighborhood operator (NO), aimed to reduce the number of settings, parses the whole current solution and for each production lot it tries to find the best position, by evaluating all newly generated solutions. Having the overall goal of decreasing the number of settings by preserving feasibility, the neighborhood operator tries to concatenate the production lots by swapping the positions of the elements. As a simple definition, a production lot represents the production between two consecutive settings on a particular machine. For instance, analyzing the sequence of production on a machine m:

Production Sequence						
Machine	shift 1	shift 2	shift 3	shift 4	shift 5	shift 6
m	12	12	34	34	34	5

one can easily identify three production lots: [12, 12] in shifts 1 and 2, [34, 34, 34] in shifts 3, 4 and 5 and [5] in shift 6.

For each of the lots identified in the solution, all possible concatenations are verified. Namely, the operator verifies whether the same product is being produced either on another machine or on the same machine and either in an earlier or later shift. Then for each such another production found, it concatenates the currently processed lot either in front or after the production lot and evaluates the obtained solutions. The best one among them is chosen and the algorithm proceeds by searching the best position for the next production lot. Hence, a pool of solutions is created, each solution being obtained from the original one by concatenating two lots. And eventually, the best non-tabu solution from the pool is selected. The following images Figure 5.3, Figure 5.4 and Figure 5.5 exemplifies how the concatenation is made. The red lot is the one being processed and the dark blue one is another production lot that produces the same item. Concatenation can be made either in front of the blue lot or after it, in the light blue positions, thus reducing the number of settings by one.

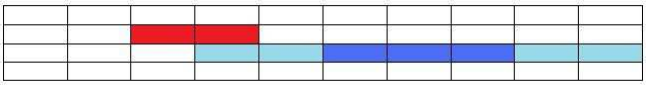


Figure 5.3: Lots concatenation

By swapping the light blue and red lots two possible solution are obtained:

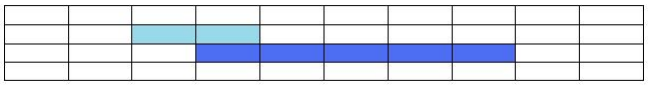


Figure 5.4: Lots concatenation

or

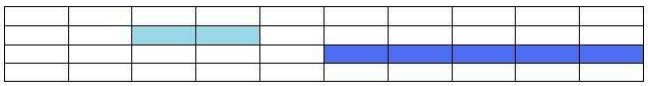


Figure 5.5: Lots concatenation

They are both evaluated and checked for feasibility and the one having the minimal objective value and is also feasible and non-tabu is added to the pool of neighborhood solutions. It might be the case that no other lot producing the same item is found, or that none of the visited solutions is feasible. Then the lot keeps its current position and the algorithm advances to the next production lot in the current solution, until all lots have been processed. The best solution from the pool is then returned by the neighborhood operator.

In a nutshell, the pseudocode of the algorithm is the following:

ReduceSettings(Solution s)

1. Neighborhood solution: aux;
2. Solution local_best;
3. while (not all production lots have been processed)
   (a) *lot* = FindNextProductionLot()
   (b) aux = DetermineBestLotPosition(*lot*, s)
   (c) local_best = BestSolution{aux, local_best}
   aux is already verified for feasibility and for holding a non-tabu status; it can also be an empty solution, in which case the local_best is not modified

4. goto (3)

5. return local_best

DetermineBestLotPosition(production_lot lot, Solution s)

1. Solution local_best

2. Parse solution s and determine other production lots *pl* that produce the same item as *lot*

   - For each production lot *pl* verify both possibilities of concatenating *lot*: before *pl* → solution s1, or after → solution s2
   - Update the local_best = Best_Feasible_and_NonTabu_Solution{s1, s2, local_best}

3. return local_best

If no concatenation can be made such that the solution remains feasible, the *DetermineBestLotPosition* procedure will return an empty solution having set a very high objective value, therefore the *local_best* is not influenced. The second neighborhood operator has the role of introducing new elements in the solution configuration, consequently new solutions will be generated.

Reduce Backlogs Operator:

Second neighborhood operator uses a swap move, by replacing products from the solution with new ones, based on the unmet demand and excessive production. Two goals are pursued by applying this operator:

- Reducing the supra-production of the items for which an overrun has been registered
- Reducing the unmet demand

In order to achieve these goals, the operator evaluates the current solution and creates a list *ud* of products having unmet demand. The elements of this list are the ones which are to enter the solution. Additionally, two lists are created, containing products that are to be replaced. Namely, a list *pic* of products registering excessive production, computed with respect to the current demand. Products belonging to the *pic* list are selected as

being the items for which a positive inventory is registered at the end of the last shift of the planning horizon. Secondly, *pif* is the list of products with supra-production, taking into account also the future demand. This list contains the items for which the inventory in the last shift exceeds the future demand after satisfying the current demand. Hence the aim is first to replace all products from *pif*, thus reducing the costs associated with the excessive production, penalized in the objective function. Nevertheless, once the *pif* list becomes empty, but there still are elements in the list of unmet demands *ud*, the approach is to replace the elements from the *pic* list, in order to reduce the costs for backlogs. Under these conditions, one defines a *redundant element* as an element e either belonging to *pif*, or in case *pif* is empty, e is an element from *pic*.

The algorithm for reducing the backlog costs proceeds by parsing the list *ud* and trying to insert each of its elements, by replacing a redundant element of the current solution. The insertion procedure is repeated until either the list of unmet demands becomes empty or there are no more redundant elements. Each insertion generates a new solution which is checked for feasibility and non-tabu status. If both conditions are satisfied, it is added to the pool of neighborhood solutions, and in the end the best one among them will be returned by the neighborhood operator. The pseudo-code for the algorithm is presented in what follows:

ReduceBacklogs(Solution s)

1. Neighborhood solution: aux
2. Solution local_best
3. Create lists ud, pif, pic
4. while (not empty ud & not empty pic)
   - foreach u in ud
   - aux = InsertProd(u, s, pif, pic)
   - local_best = BestSolution{aux, local_best}

   aux is already verified for feasibility and for holding a non-tabu status; it can also be an empty solution, in which case the local_best is not modified
5. goto 4
6. return local_best

The important role is assigned to the procedure *InsertProd*, which takes as parameters the element to be inserted u, the current solution s, and the lists of redundant elements

pif and *pic*. And it returns a new solution obtained by replacing one redundant element with u. *InsertProd* aims to insert u in the best position, namely such that the obtained solution has the minimal objective value. Thus, the current solution s is parsed and each element is verified for redundancy. Once a redundant element is found, it is replaced by the element u and the feasibility of the newly generated solution is checked. Only feasible solutions compete for becoming the next candidate solution. By replacing one redundant element with u, a set of new solutions is created and the best one among the feasible and non-tabu ones is selected to be returned. Then, the element u is eliminated from the list of unmet demands. The pseudo-code of the *InsertProd* algorithm is depicted in what follows:

InsertProd(Element u, Solution s, List pif, List pic)

- for m = 1, M
- for t = 1, T
    1. Current solution: s , restore Y
    2. if (is redundant element Y[m,t] in solution s)
    3. replace: Y[m,t] = u
    4. Update the local_best = Best_Feasible_and_NonTabu_Solution{s, local_best}
- endfor
- endfor
- update lists ud
- return local_best

where M is the total number of machines, T is the number of shifts, Y is the production matrix and local_best is an auxiliary variable to determine the best feasible and non-tabu solution among the visited ones.

Note that when a new element is inserted in the configuration of the current solution, the operator does not check the due time of the processed demand. Nevertheless, it is the duty of the neighborhood operator to return the solution that has the minimal objective function among the evaluated ones. Additionally, the main goal is to introduce the required item in the next current solution, as its position will also be changed by the ReduceSettings neighborhood operator, until the best one is found. On the other hand, the ReduceBacklogs operator complements the ReduceSettings operator. As some items may

be produced in the correct quantity on a machine, but since the ReduceSettings operator moves the production lot on a machine with a different capacity, either unmet demand or excessive production may be registered. Hence, this unmatch between production and request is corrected by the ReduceBacklogs operator. However, the ReduceBacklogs operates in small steps, by changing the production of only one shift of a redundant element with the production of a new item, which registers unmet demand. Thus it does not take into consideration the capacity of the machine and whether the full required quantity is produced for the newly inserted items. The aim is to fine-tune the search by inserting a new element and to create a new solution slightly different than the current one.

The need of a more powerful operator is met due to the neighborhood operator ReduceBacklogsReplaceLots. It operates in a similar manner to the previous neighborhood operator, but instead of replacing only one shift of a redundant element, it replaces the full lot with the new item. Again, all possible insertions are analyzed and the feasible, non-tabu solution that performs best is returned.

The above presented neighborhood operators have as direct target to reduce the costs, either settings, backlogs or unmet future demand costs. However, for some configurations of the solution, they are to restrictive and are not capable of generating new solutions. In order not to remain entrapped in the basin of attraction of one local optima, adjuvant neighborhood operators are employed. In contrast to the previous neighborhood operators, which are called in every Tabu Search iteration, these NO are called less frequently.

ExtendShiftLots Operator:

The *ExtendShiftLots* NO has the role to parse the current solution and for each lot to create four new solutions by shifting or extending it one position to the right or one position to the left. Among the four generated solutions the best performing, feasible and non-tabu solution is returned. By shifting the lot one position to the left, one understands that the lot on the left-hand side of the lot being processed is shrank one position and the right-hand side lot is extended one position, as in the following Figure 5.6:

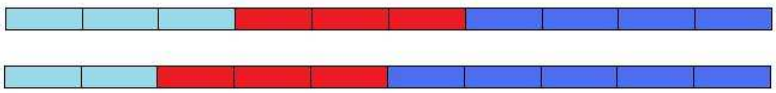


Figure 5.6: Shift lot one position to the left

Similarly for shifting the lot to the right. An extension of the lot is translated into the

fact that the currently processed lot overwrites one shift of the production of the lot from the left or right, respectively, as depicted below Figure 5.7:

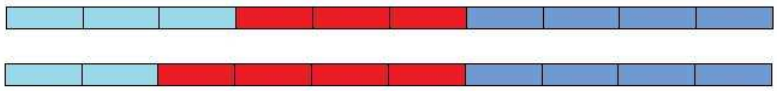


Figure 5.7: Extend lot one position to the left

Hence, the pseudocode of the *ExtendShiftLots* NO is:

ExtendShiftLots (Solution s)

1. Neighborhood solutions: s1, s2, s3, s4
2. Solution local_best
3. for each lot l in s
   - s1 = ShiftLotLeft(l,s)
   - s2 = ShiftLotRight(l,s)
   - s3 = ExtendLotLeft(l,s)
   - s4 = ExtendLotRight(l,s)
   - local_best = BestSolution{s1, s2, s3, s4, local_best}
     the operators ShiftLotLeft(l,s), ShiftLotRight(l,s), ExtendLotLeft(l,s) and ExtendLotRight(l,s) are responsible to return a solution only if it is feasible and non-tabu; otherwise an empty solution will be returned that does not influence the local_best
4. return local_best

ExchangeLotProduct Operator:

The fifth NO *ExchangeLotProductOperator* has the role of introducing new items in the solution, independent of the demand. Hence, it parses the solution and for each lot, it substitutes the item being produced with another item and generates new solutions. Among the feasible and non-tabu ones, the best performing solution is returned. The pseudocode is given in what follows:

ExchangeLotProductOperator(Solution s)

1. Neighborhood solution: aux
2. Solution local_best
3. for each lot l in s
4. for each item j in J
   - Replace item produced in lot l with j and obtain solution aux
   - Update the local_best = Best_Feasible_and_NonTabu_Solution{aux, local_best}
5. return local_best

SwapLotsOperator Operator:

Another approach aims at swapping the items produced in two different lots. Based on this idea two NO have been developed, one which makes all possible swaps and returns the best performing, feasible and non-tabu solution. And a second one, which performs a swap only if the item produced in one lot would be produced more efficiently on the machine of the other lot.

Hence, the pseudocode is:

SwapLotsOperator(Solution s)

1. Neighborhood solution: aux
2. Solution local_best
3. for each lot *l1* in s
4. for each lot *l2* in s
   - (*) if (capacity[m1][p1] < capacity[m2][p1] || capacity[m2][p2] < capacity[m1][p2])
     m1 - machine *l1*, p1 - item *l1*; m2 - machine *l2*, p2 - item *l2*
   - aux = swap(*l1* $\leftrightarrow$ *l2*)
     l1 := item produced in *l2* and *l2* := item produced in *l1*
   - Update the local_best = Best_Feasible_and_NonTabu_Solution{aux, local_best}

5. return local_best

Including the capacity condition for swapping the lots, the NO *SwapLotsCapacityOperator* is described.

5.3.2 Tabu Search Algorithm

The search algorithm implements the method of the Tabu Search meta-heuristic in its classical form. Thus it follows the trajectory of an initial solution which is generated using the construction algorithm described in the section *Initial Solution Construction*. Applying the above presented neighborhood operators, new solutions are generated and the best one which is not tabu is selected to be the current solution of the next iteration. Hence these steps repeat until the termination criteria are satisfied. The following subsections are describing each of the key elements of the TS method in more details.

Tabu list: Making use of the recency-based memory, one stores an identification element associated to each recently visited solution in a list called *tabu list*. In the current implementation, the *explicit* form of memory is preferred. Namely, the whole recently visited solutions are stored, since the amount of memory required for storing the production matrix is less than 1KB, even for complex instances of the production planning problem. Therefore a solution is set tabu if it has been visited before, and it keeps the tabu tenure for a fixed number of iterations, determined by the length of the tabu list. This is established by a meta-parameter to be *TLen* and each visited solution is added to this list on a *first-in-first-out* principle.

Aspiration criteria: The aspiration criterion most generally adopted is to accept a tabu solution if it performs better than any solution visited so far. But according to the above presented definition of the elements of the tabu list, a solution cannot simultaneously have a tabu tenure and surpass in performance the best found solution.

Termination criteria: There are two termination criteria invoked for determining the running time of the search. Firstly, the search is stopped if the solution cannot be improved for a determined number of iterations *NonImp*, and secondly, if the number of iterations reaches the maximum established *maxTabu*. Both *NonImp* and *maxTabu* are meta-parameters for guiding the search.

Meta-parameters: As presented, the short-term memory based strategies are guided by a series of meta-parameters. The *maxTabu* meta-parameter adjusts the balance between a high quality solution and the running time of the algorithm. Large values of these parameter allow the search to visit an increased number of solutions, thus increasing the chances of finding better ones. On the other hand, time is an important factor and the *maxTabu* parameter can determine the algorithm to stop after a reasonable amount of time.

In direct connection with *maxTabu* is the meta-parameter *NonImp* which stops the search if the solution cannot be improved for *NonImp* number of iterations. Moreover, *TLen* which determines the length of the tabu list. The value of *Tlen* should be adjusted such that the search is not too much restricted, for high values, but still, restricted enough such that it can escape local optima.

Search algorithm: Following, the pseudo-code of the Tabu Search algorithm is given below:

TabuSearch()

1. Current Solution: currSol = ConstructInitialSolution()
2. Neighborhood Solutions: s1, s2, s3, s4, s5, s6, s7, neighSol
3. Solution: best
4. Add the current solution currSol to the tabu list
5. while (not Termination_Criteria)
    - Every 5 iterations apply only the NOs:
    - currSol = SwapLotsCapacityOperator(currSol)
    - currSol = ReduceBacklogsReplaceLots(currSol)
    - In all other iterations:
    - s1 = ReduceBacklogs(currSol)
    - s2 = ReduceBacklogsReplaceLots(currSol)
    - s3 = ReduceSettings(currSol)
    - Every 3 iterations:
    - s4 = ExtendShiftLots(currSol);
    - Every 5 iterations:

- s5 = SwapLotsOperator(currSol);
- s6 = ExchangeLotProductOperator(currSol);
- neighSol = BestSolution{s1, s2, s3, s4, s5, s6}
- if (neighSol is empty OR every 50 iterations)
- Perform a diversification step: s7 = DiversifyCurrent(); as explained in the next section
- neighSol = BestSolution{s7, neighSol}
- currSol = neighSol
- Add currSol to the tabu list
- best = BestSolution{best, currSol}

6. goto 5
7. return best

5.3.3 TS Advanced Strategies

Short-term memory based strategies provide good results, however they can be improved by means of long-term memory use. Information acquired during the searching process can be a very useful guideline in exploring new regions or exploiting the interesting ones. As a form of long-term memory, frequency measures are registered for the purposes of intensification and diversification. In this respect, an attribute is defined as the tuple (MachineID, ShiftNr, ProductID) and the frequency measure represents the number of solutions in which this attribute was included. Hence, for each generated solution, the matrix of frequencies is updated. One may notice that, in contrast to the case of recency-based memory, this is a form of attributive memory. On the other hand, explicit memory is used for keeping track of the elite solutions encountered throughout the search. A number of selection strategies have been described in literature for selecting the elite solutions, as explained in Chapter IV. In the current implementation, a list of fixed length is maintained, always the solution performing worst from the list being replaced if a solution which performs better is visited. The length is established by means of a meta-parameter *elLen*. The main purpose of the elite list is to restart and intensify the search in the region of these solutions, nevertheless it has a role also in achieving diversification, as one will see in what follows. The intensification steps are performed after the short-term based TS algorithm finishes its execution. In contrast, the diversification step is integrated in the short-term TS algorithm, to be performed when a critical event appears,

such as an empty neighborhood. Moreover it diversification is performed regularly to add new solutions to the neighborhood of the current solution, and repeated as a separate phase when the TS algorithm finished its execution.

Intensification

As soon as the searching strategies based on the short-term memory lose their efficiency, the next step is to perform intensification in the most promising discovered regions. Three strategies for achieving intensification have been implemented for the current production planning problem.

The first approach represents one of the extreme cases of path relinking, namely, it starts with an empty initiating solution and has as guiding solutions all the elite solutions memorized in the list. The goal is to create an new candidate solution, by including attributes from the guiding ones. Therefore, the production matrix is constructed by assigning to each entry Y[m,t] the item j for which the attribute (m,t,j) is included in the greatest number of elite solutions: m represents the machine index, t the shift and j the item. If the generated solution is feasible and non-tabu, the search is then resumed from this point and a new trajectory is explored. The newly generated solution may not be evaluated to a good objective function value. Nevertheless, it includes high quality attributes which allow the neighborhood operators to create new high quality candidate solutions, starting from the current configuration. The list of elite solutions is continuously updated and the next step of intensification may be based on new high quality solutions discovered during the previous intensification phase. The quality of the generated solution is highly influenced by the value of *elLen*. Better results have been obtained for small values of this parameter, as the power of influence of high quality solutions is not diminished by medium quality ones.

The second approach restarts the search from the encountered elite solutions. A number of regions is explored in this intensification phase, by resuming the search from a number of different points in the search space, which historically proved to be good. This intensification step is always performed, since the starting solutions are always feasible. The aim is to exploit the neighborhood of the promising solutions in the hope of finding better performing ones.

The third intensification step, starts from the current solution and creates a pool of new solutions by changing one attribute at a time. Thus an attribute of the current solution is substituted by the most frequently corresponding attribute in the elite solutions. The best *elLen* solutions are selected and the search is restarted from these solutions.

Diversification

If by applying short-term memory based strategies and performing intensification, higher quality solutions cannot be reached, then new regions of the search space must be discovered. Hence, a diversification step is performed regularly in order to enlarge the neighborhood of the current solution, either to overcome the shortcoming of the neighborhood operators when they cannot escape a local optima or to guide the search towards new regions.

The diversification step included in the short-term based TS algorithm starts from the current solution, and similar to the intensification phase, replaces some of the attributes with those found in the elite solutions, taking care that feasibility is preserved. In contrast to the intensification step, more than one attribute is replaced and the substituting attributes are the ones less frequently used in the elite solutions. In this manner, the newly generated solution is created starting from the current one, therefore it remains in the neighborhood of the current solution, and additionally, it introduces some new attributes, allowing the search to escape a local optimum.

Also, diversification steps are performed as advanced strategies employed after the short-term memory based TS finishes its execution.

The first strategy is to build a new initial solution and restart the search. Here is when the frequency based memory comes into the scene. As the aim is to reach new or less explored regions, one will construct a new initial solution from the less frequently used attributes. Therefore, the matrix of production of the next current solution will be built by assigning to each entry Y[m,t] the index of the item j for which the attribute (m,t,j) has the minimal frequency measure: m is the machine index, t the shift and j the item index. A new solution will be obtained, including attributes that will guide the search towards new areas. This step is very important for the vigor of the search, permitting the algorithm to escape the basin of attraction of the discovered local optima and to search for better performing solutions into new regions. Rather than randomly selecting a new starting solution, long-term memory is used for guiding the search, as one of the key elements of the Tabu search method. The searching process restarts from the newly constructed solution only if it is feasible.

Similarly, the second strategy creates a new solution from the less frequently used attributes in the elite solutions, rather than in all the solutions visited during the search process. The reason lays in the fact that these regions might not yet have been fully explored. Therefore a new candidate solution will be constructed by assigning to an entry Y[m,t] the item j that has the lowest number of occurrences in the elite solutions in the

same position [m,t]. If the generated solution is valid, the search is resumed and a new region is explored.

All in all, short-term memory based TS performs well and good quality solutions are generated. Nevertheless, higher quality ones are obtained by performing advanced search strategies, like intensification and diversification, which make use of the information acquired during the first step of the search.

Chapter 6

Documentation of Results

Although simple in its basic idea, Tabu Search algorithm, together with the advanced associated strategies, proved to be very efficient in generating quality solutions for the considered production planing problems. The presented method has been tested on six instances of PP problems of medium and high complexity and experimental testing showed that good and very good results have been obtained. Complex instances of the problems are generically called Big Problems and the algorithm creates a production plan with a time horizon of 15 shifts, for 21 machines able to produce 272 types of products. The small-medium instances are called Toy Problems and the number of shifts considered remains the same, 15, but the planning is built for 6 machines that produce 50 types of products.

In what follows, one will find detailed results for each of the presented problems and plots depicting the trajectory of the solution throughout the whole searching process. In contrast to other optimization techniques, the solutions generated by the TS method do not converge to the best found optimum. This is a consequence of the fact that non-improving steps are accepted in Tabu search in order to allow the algorithm to escape local optima, but also due to the diversification phases. Thus the best found solution may be visited in any phase of the searching, and the algorithm will continue until the termination criteria are satisfied. The graphics also illustrate the trajectory of the best found solution, which will have a monotonously decreasing trajectory, including some phases of stagnation. The solutions were generated by running the TS method for approximately one hour for each instance of the production planning problem. In addition to the plots depicting the solutions' trajectory, the best obtained results for each problem have been documented in a table indicating the value of the objective function and the costs (settings, backlogs, future unmet demand and inventory violation) associated to the generated production plan.

Finally, the results obtained by the TS method are compared to the ones generated by an exact method, Mixed Integer Programming, and another metaheuristic, GRASP. For a relevant comparison, all three algorithms generated the results after performing a one hour time search.

Next, the visited solutions for each of the solved production planning problems will be plotted as a graph having on the Ox axis the number of the current iteration and on the Oy the value of the objective function of the solution visited in the corresponding iteration. The result for each problem is depicted in two pictures. The first one illustrates the trajectory of both the current and best solution during the whole search. And the second one shows a zoom of the trajectory of the current and best solution in an interval close to the value of the objective function corresponding to the best found solution.

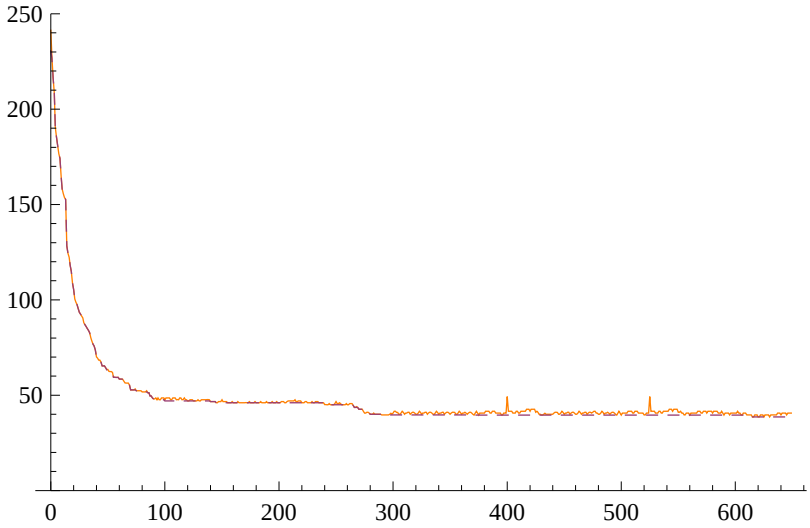


Figure 6.1: Plot Problem 1

Figures 6.1 and 6.2 plot the trajectory of the solution for the problem 1, created by the Tabu Search algorithm. The orange graph represents the trajectory of the current solution, while the dashed line indicates the trajectory of the best solution. One can easily notice that in the beginning both the current and the best solution have a very fast descend. This is due to the fact that the configuration of the initial solution allows the construction of a very large neighborhood, such that one can always find a candidate solution that outperforms the current one. Therefore the current and the best solution follow almost the same trajectory. Having reached a certain configuration, the solution cannot be improved significantly any further. One can notice a plain trajectory of the best solution, with only small decreasing steps. On the other hand, the zoomed image pictures the trajectory of the current solution with intensification steps, where the solution remains in the limits of the same interval of the values of the objective function and diversification steps, where different solutions are accepted, although they perform worse

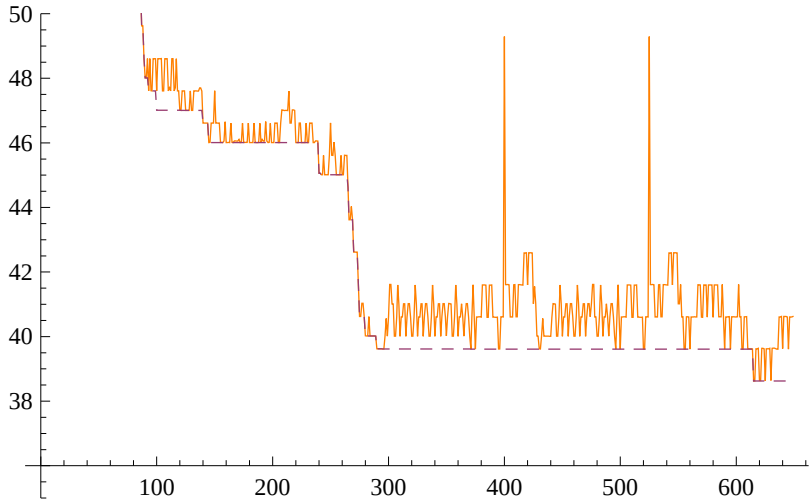


Figure 6.2: Plot Problem 1 Zoom

than the current one. The best obtained solution has the value 38.625163 and it was obtained in approximately one hour.

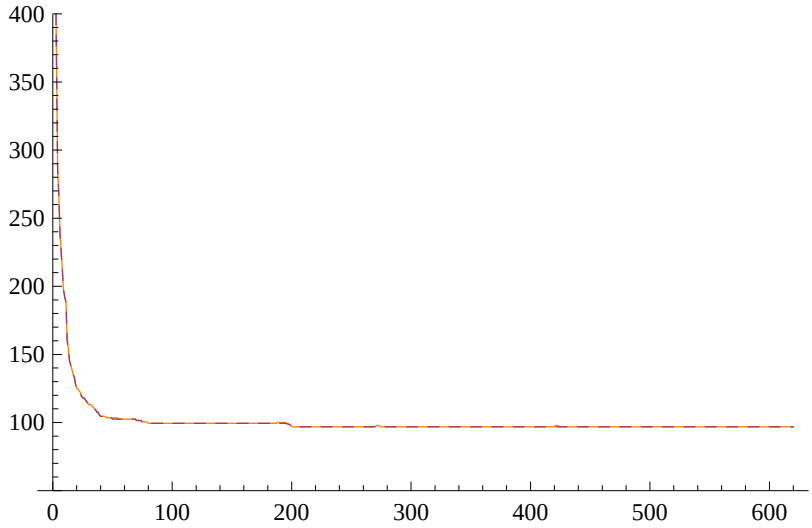


Figure 6.3: Plot Problem 2

The trajectory of the solution in the case of the second problem is depicted in Figures 6.3 and 6.4. As the picture indicates, the current and best solution follow again a very fast descend in the beginning, as the configuration of the initial solution allowed a high number of improving moves. Nevertheless, for a high number of iterations, the improvements of the best known solution are very small. The zoomed image shows that the algorithm has been restarted several times from different points, as the best solution was not modified for a predefined number of iterations. However the search has been again attracted by the same local optima. This can be explained either by the fact that the found local optimum represents a global point of optimality, or that the diversification phase was

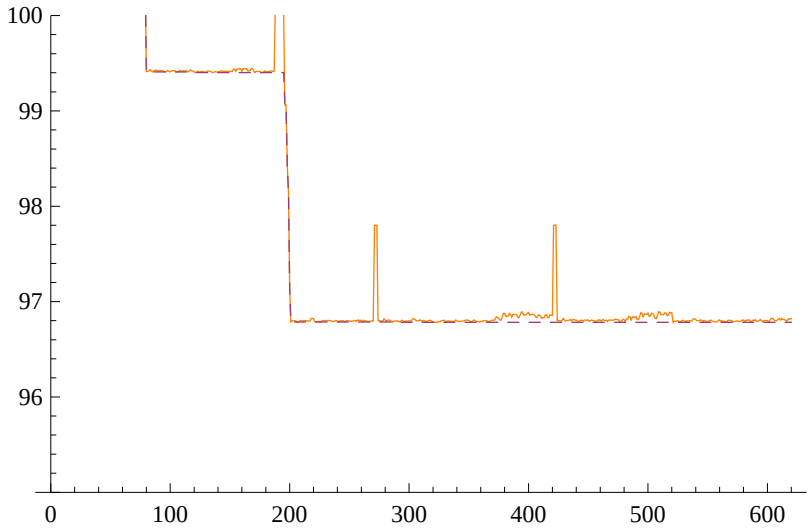


Figure 6.4: Plot Problem 2 Zoom

not generating a solution different enough to escape the basin of attraction of the visited local optimum. Therefore, although the algorithm continued the search for one hour, in 600 iterations, the solution found in the approximately 200th iteration was not improved further than the value 96.783547.

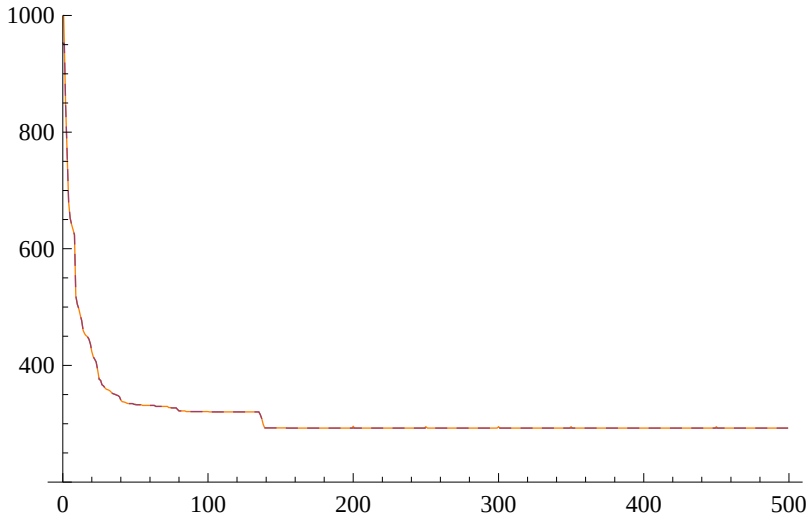


Figure 6.5: Plot Problem 3

Very similar to the second problem, the plot of problem number 3 in figures 6.5 and 6.6 indicates a very fast improvement of the solution in the first approximately 150 iterations and a stagnation of the best solution for the last iterations. The zoomed picture illustrates that the current solution was always attracted in the neighborhood of the same local optima, despite the diversification steps performed every 50 iterations. Hence the best achieved result is 292.482963, which again could not be improved although the TS method

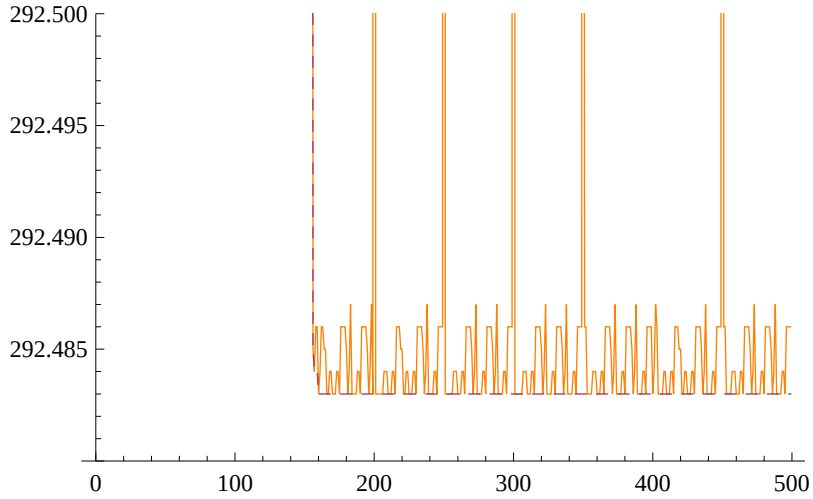


Figure 6.6: Plot Problem 3 Zoom

was searching for one hour.

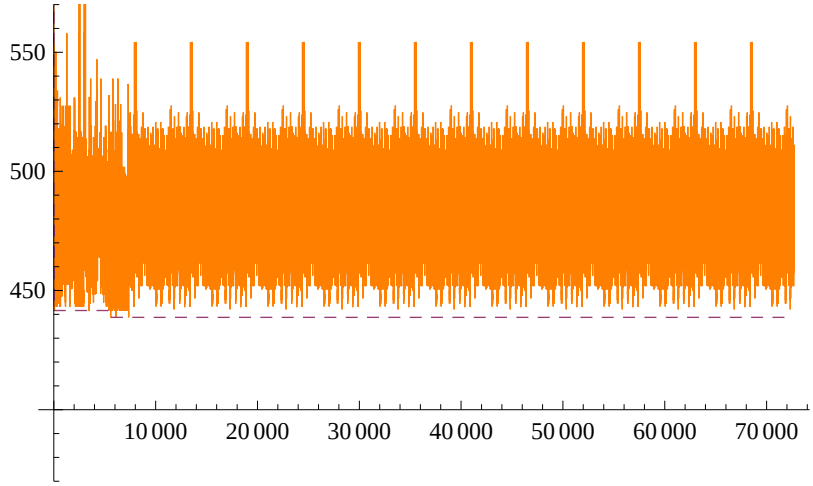


Figure 6.7: Plot Problem 4

Figures 6.7 and 6.8 illustrate the results of the fourth problem. The trajectory of the current solution indicates that the algorithm has explored diverse regions. The intensification phase in the neighborhood of the elite solutions leaded to the best known optimum. After reaching the threshold 438.78291, the value of the best known solution was already determined, although the algorithm continued to explore different areas of the search space. A very large number of iterations has been performed in the one hour time interval. In contrast to the previous three problems analyzed, the size of this problem and the next two is considerably decreased. Hence the metaheuristic can perform a more detailed search. TS can visit a highly increased number of candidate solutions in the same fixed CPU time.

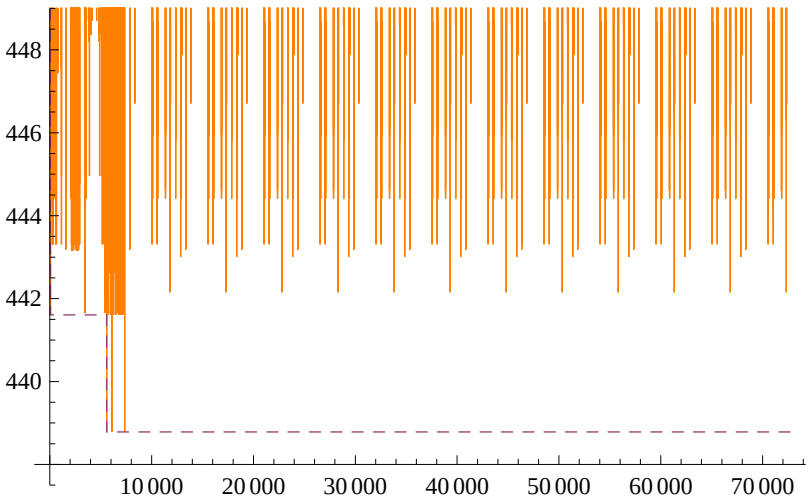


Figure 6.8: Plot Problem 4 zoom

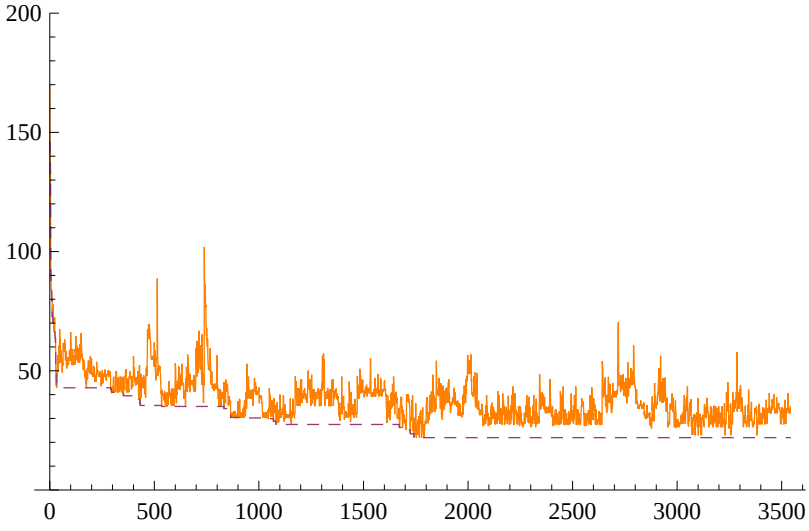


Figure 6.9: Plot Problem 5

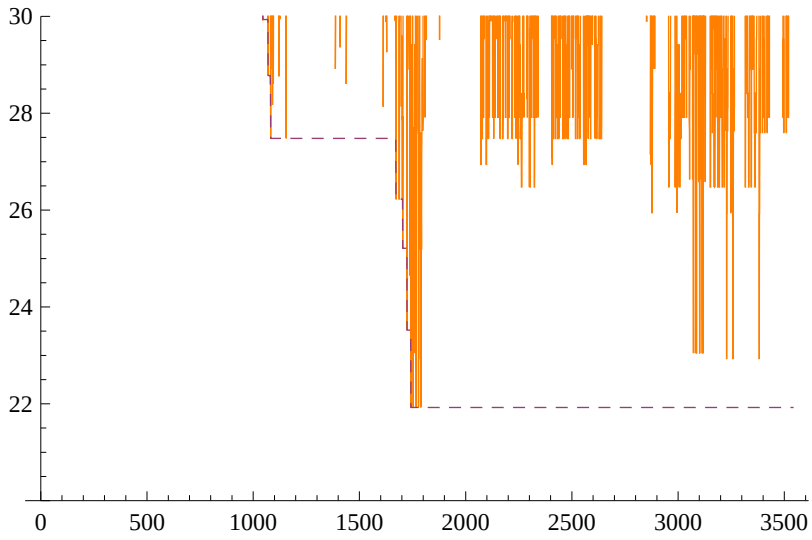


Figure 6.10: Plot Problem 5 zoom

Figures 6.9 and 6.10 clearly picture the intensification and diversification steps visible in the trajectory of the current solution. The intensification step started in the approximately 1700th iteration in the neighborhood of the best known solution known until that moment, was very successful in discovering better and better solutions in the close neighborhood until the value 21.922567 has been reached. In the next over 1500 iterations new diversification and intensification steps have been performed, nevertheless no better performing solutions have been visited.

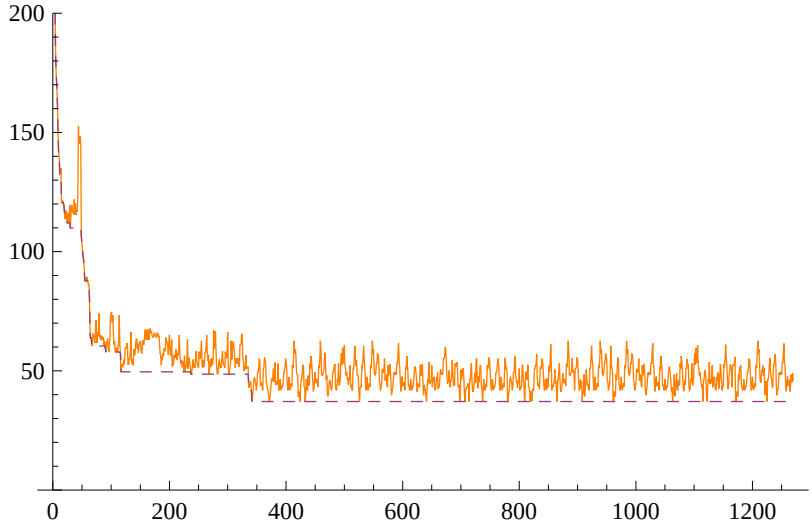


Figure 6.11: Plot Problem 6

The sixth problem starts with an initial solution having a high value of the objective value, which is fast decreased in the first iterations of the search. As shown in Figures 6.11 and 6.12, the current solution follows a non-monotonous trajectory even in the beginning,

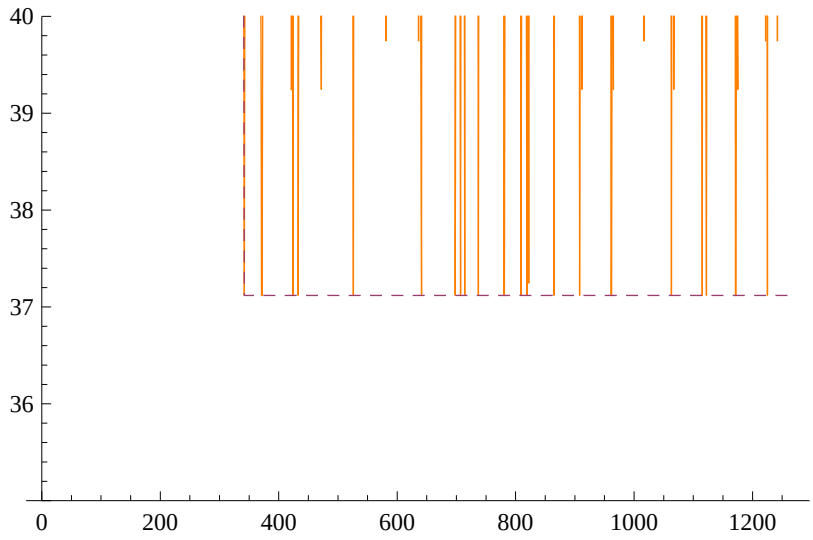


Figure 6.12: Plot Problem 6 zoom

due to some diversification phases employed in the search and as the search continues the alternative intensification and diversification steps lead the current solution to reach the value of the best solution very often, however it cannot be further improved. Again, this might be a sign that the local optimum represents a global one or that it has a very large attraction basin which cannot be escaped by the proposed TS strategies. The best found solution has a value of 37.119038, reached in the 350th iteration out of more than 1200.

The numerical results for each problem are given below. The algorithm run for one hour and these are the best results obtained in this interval of time. However, the solutions may have been found in any phase of the search, but the algorithm continued to visit other candidate solutions in the hope of finding better ones. The meta-parameters have been fixed, such that there is a best operating set for the big problems and a set working proper for the toy problems. Although various values of the parameters would give better results if adjusted for each individual problem, the fine-tuning of the parameters may be time consuming. Hence, a set of parameters that operates well on every instance of the problem is preferred.

Each row gives details for the problem specified as the entry in the first column. The second column indicates the value of the objective function and the 3rd one - the time of the search. The costs represent the elements building the objective function: namely the setting costs, the costs inferred for no meeting a current demand, called backlogs; additional costs are taken into consideration for registering unmet future demand, but also costs for producing in excess are imposed, and the latter ones can be found in the column InV. The last three of the columns represent the parameters' setting, for guiding the search. Therefore, the tabu list has a length of 10 solutions, for the three big problems

and 15 for the toy problems. Toy problems 2 and 3 provide similar results for values of Tlen between 10 and 15. The number of iterations of the meta-heuristic, called Nr of Tabu iterations, is established to 100 for the Big problems, and 5000 for the Toy problems. The algorithm is also programmed to stop in case no improvement can be made to the known best solution for 50 iterations for the Big problems and 500 iterations for the Toy problems.

Problem	Objective Function Value	CT (min)	Costs				Parameters		
			Settings	Backlogs	UFD	InV	TLen	Nr of TI	Max non-imp
Big 1	38.625163	60	38	0	0.62	0	10	100	50
Big 2	96.783547	60	47	49.48	0.29	0	10	100	50
Big 3	292.48318	60	49	243.03	0.45	0	10	100	50
Toy 1	438.78291	60	10	428.56	0.62	0	15	5000	500
Toy 2	21.922567	60	17	3.95	0.97	0	10-15	5000	500
Toy 3	37.119038	60	19	17.18	0.93	0	10-15	5000	500

Where

- CT = Computing Time
- UFD = Unmet Future Demand
- InV = Inventory Violation
- TLen = Tabu List length
- Nr of TI = Nr of Tabu iterations
- Max non-imp = Maximal nr of non-improving iterations

Additionally, for the toy problems the intensification phase restarts the search from all the elite solutions memorized during the search. On the other hand, in the case of the big problems, the intensification phase consists in a throughout exploitation of the neighborhood of the best known optimum.

The results obtained after one hour running time, are compared to the ones obtained by two other methods. Namely, the exact method MIP and another heuristic method, Greedy Randomized Adaptive Search Procedures (GRASP), presented in the paper [14].

The first method TS is compared to, is MIP, an exact method which can also provide information regarding the quality of the achieved result, in comparison to the global optimum. Moreover, if allowed to run for sufficient time, it can even provide a globally optimal solution. However the required time might be a matter of few hours, for small instances of the PP problem, to days, for complex instances. The results provided by the two optimization techniques, TS and MIP, are similar from the point of view of the objective value. In four out of six cases TS outperforms MIP, while in the other two situations, MIP manages to generate better results in the one hour time interval.

The second evaluated meta-heuristic, GRASP is an optimization technique which involves the random element. Hence, for the same input, and the same configuration of the parameters, different results are obtained in each run. Therefore, an average of these results is considered for the comparison. Additionally, two variants of the GRASP algorithm have been implemented. The sequential GRASP and a parallel version of the greedy randomized method. The result generated by our TS implementation will be compared to the ones obtained by the sequential GRASP and also with those provided by the parallel GRASP. As the following table illustrates, TS outperforms sequential GRASP in three out of six problems, for one case they generate the same solution, which also represents the global optimum, proven by MIP and in the other two cases sequential GRASP performs better than TS. Regarding the parallel GRASP, the situation is reversed. Thus TS generates two results of higher quality than GRASP, for one problem both algorithms generate the same solution and in three cases, parallel GRASP outperforms TS.

Comparison	TS	MIP	GRASP average Seq	GRASP average Par
Big 1	38.62	42.53	39.32	38.95
Big 2	96.78	96.18	98.41	96.20
Big 3	292.45	240.61	246.70	247.53
Toy 1	438.78	439.94	438.75	438.75
Toy 2	21.92	24.91	23.52	22.21
Toy 3	37.11	37.13	29.67	29.82

An illustrative view of the above table is depicted in the Figure 6.13.

As one can see, all three algorithms generate very similar results, from the point of view of their objective function value. However, in the case of Big 3 problem, TS does not have time to escape the local optimum. The graph of the trajectory of the current

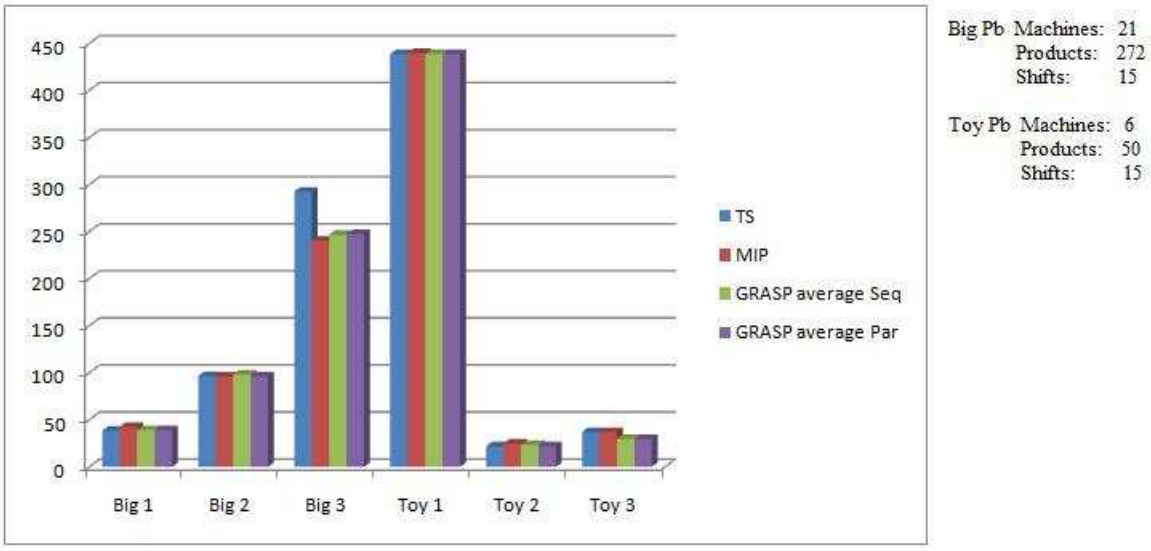


Figure 6.13: Comparison

solution indicates that better results can be obtained if the length of the tabu list would be dynamically adapted to the landscape of the region being explored.

Chapter 7

Conclusions

Due to its economical importance, production planning has represented an interesting research area and various approaches have been proposed for discovering high quality solutions. However, for complex instances of the production planning problems, a heuristic, or more specifically, a metaheuristic method is preferred, rather than an exact one. Among them, in the current thesis, a Tabu Search based algorithm is proposed for tackling a real-life production planning problem from the automotive supply industry. The results of our method are to be compared to those obtained by non-deterministic heuristic method, GRASP, and by an exact method, Mixed Integer Programming.

Having analyzed the current state of the art and based on the previously achieved results concerning the PP problem, the implementation of a version of the Tabu Search method has been presented in detail. An algorithm for constructing an initial solution is described. The advantages of the construction algorithm are given by the fact that it will always generate a feasible solution in a very short CPU time. Additionally, this solution represents a valid planning which can stand as the initial solution of any other optimization technique. The core of the thesis lays in the Tabu Search algorithm for which both short-term memory based strategies, as well as advanced, long-term memory based strategies have been developed. The production planning problem proposed for solving considers a time horizon of five days, with both current and future demands, with varying capacity with respect to product type and a large set of complex constraints. Hence, feasibility has been a constant and important concern in all steps of the searching process. Moreover, the objective function was designed to optimize four aspects simultaneously. Namely to reduce the costs associated to the production process, settings costs, and the costs related to not meeting the demands, backlog costs for the current demands, the costs for not meeting the future demand. Also inventory costs are taken into account for a supra-production that exceeds the current and future demand.

Starting from the initial solution, a trajectory has been built, by means of seven neighborhood operators. Three of them have been designed with the aim of reducing the costs included in the objective function. Additionally, four more operators have been implemented for enlarging the neighborhood of each solution. Hence they ensured the vigor of the algorithm, and together with the memory based strategies they allowed the search to escape local optima. The neighborhood construction represents a very important aspect for the performance of both the TS and the neighborhood search. Furthermore, the efficiency of the algorithm has been enhanced by means of advanced strategies, that make use of the intermediate and long-term memory, which keeps information collected during the searching process. In this respect, three different intensification and three diversification methods have been proposed to contribute to the interplay between exploitation of interesting areas and discovering of new ones, in order to intelligently explore the search space. Ideas from the path relinking strategy have been borrowed and included in the intensification methods. Although a basically simple technique, the TS is guided by a collection of metaparameters. Based on experimental results, the best operating parameters for the Tabu Search have been established. Therefore, for a particular production planning problem high quality results are obtained by running the algorithm with the standard values of the parameters. In addition, fine tuning is also possible, for boosting the performance of the TS for the specific problem.

Being a metaheuristic solution method, optimality of the obtained results cannot be guaranteed. However, very good solutions are generated after running the algorithm for only one hour. Due to more intensification and diversification phases, larger running times allow the algorithm to discover even higher quality solutions. The proposed method has been tested on six instances of production planning problems, three medium and three complex ones. Results from extensive testing indicate that our metaheuristic is effective in producing very good solutions to all these problems. As a remark, the short-term memory based Tabu Search already provides production plans of a good quality for all of the considered problems, even after a short running time. But applying the advanced strategies has as consequence a significant improvement in most of the cases. Tabu Search may take considerable more CPU time than a simple heuristic, however the resulting solutions are better than those obtained by a local search, since this method cannot escape local optima. In comparison to the MIP solutions, the exact method is able to find the global optimum for the production planning problems, but in a larger amount of time. However, TS tends to perform as better as MIP or even outperform it for some instances of the PP problem. Nevertheless, in some situations MIP provides better solutions. The comparison with the GRASP metaheuristic reveals that the average of the results obtained by the greedy randomized method and TS are very close in quality. In contrast to the GRASP metaheuristic, Tabu Search is a deterministic method, hence

the same results will be generated for each running of the algorithm. On the other hand, GRASP may for some runs provide better results and for other runs, solutions which perform worse than the average.

Overall, the Tabu Search technique represents more an engineering approach to complex and large size optimization problems, than an elegant and simple mathematical algorithm. However, even basic TS strategies proved to yield high quality results. Additionally, typical TS advanced strategies boost the performance of the algorithm. It represents a challenge to develop heuristics for complicated models and solve real-life production planning problems with multiple and complex constraints. But TS was a practical and successful approach in tackling these problems. Apart from retaining the advantages of any metaheuristic, through the metaparameters, the algorithm gains flexibility and adaptability to diverse instances of the production planning problem. One can adjust the values of the guiding parameters such that high quality results are obtained for any particular problem. And in many cases these results are provided after a relatively short running time. Another advantage is that the metaheuristic can be easily extended to englobe more neighborhood operators or handle even more complex models.

All in all, the aim of this thesis is to analyze previous approaches for modeling and solving the production planning problem and to give an alternative, based on the Tabu Search method. The metaheuristic is built around the described neighborhood operators specifically designed for this particular real-life PP problem.

Bibliography

[1] http://en.wikipedia.org/wiki/johnson's_rule. June 25, 2009.

[2] http://geneticalgorithms.ai-depot.com/tutorial/overview.html. June 25, 2009.

[3] RISC Software GmbH. June 25, 2009.

[4] Davenport A. and Kalagnanam J. Scheduling Steel Production using Mixed-Integer Programming and Constraint Programming. *Book Principles and Practice of Constraint Programming*, 4741/2007(1):64–76, 2007.

[5] Staggemeier A.T. and Clark A.R. A Survey of Lot-Sizing and Scheduling Models. In *23rd Annual Symposium of the Brazilian Operational Research Society (SOBRAPO) Campos do Jordão*, Brazil, 2001.

[6] Fahimnia B., Luong L.H.S., and Marian R. M. Modeling and Optimization of Aggregate Production Planning - A Genetic Algorithm Approach. In *Proceedings of World Academy of Science, Engineering and Technology*, volume 15, 2006.

[7] Dr. Stainko R., Contact person from the RISC Software Company. Personal Communications.

[8] Kimms A. Drexl A. Lot sizing and scheduling - survey and extensions. *European Journal of Operational Research*, 99(1):221–235, 1997.

[9] Glover F. and Laguna M. *Tabu Search*. Kluwer Academic Publishers, Massachusetts, 2002.

[10] Glover Fred and Laguna M. Tabu Search atricle.

[11] Kung L.C. and Chern C.C. Heuristic Factory Planning Algorithm for Advanced Planning and Scheduling. *Computers and Operations Research*, 36(9):2513–2530, 2009.

[12] Wagner M. and Whitin T.M. Dynamic version of the economic lot size model. *Management Science*, 5(1):89–96, 1958.

[13] Garey M.R. and Johnson D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, 1979.

[14] Alexandrov P. GRASP with Path Relinking for a Discrete Lotsizing and Scheduling Problem with Non-trivial Constraints. Master's thesis, Johannes Keppler University of Linz, July 2009.

[15] Production Planning. June 25, 2009.

[16] Raza S.A., Akgunduz A., and Chen M.Y. A Tabu Search Algorithm for Solving Economic Lot Scheduling Problem. *Journal Heuristics*, 12(1):413–426, 2006.

[17] Graves S.C. Manufacturing Planning and Control, November 1999. Massachusetts Institute of Technology.

[18] Weisstein W.E. NP-Hard Problem. June 25, 2009.